12-2025

Multi-Agent Robotaxi Dispatch Coordination in a Real-World Simulation – Optimizing Rider Assignment, Rebalancing, and Charging Using Battery-Dependent Rewards and Welfare Maximization

Paden Thompson
Utah State University

MULTI-AGENT ROBOTAXI DISPATCH COORDINATION IN A REAL-WORLD SIMULATION – OPTIMIZING RIDER ASSIGNMENT, REBALANCING, AND CHARGING USING BATTERY-DEPENDENT REWARDS AND WELFARE MAXIMIZATION

by

Paden Thompson

A thesis submitted in partial fulfillment
of the requirements for the degree

of

MASTER OF SCIENCE

in

Computer Science

Approved:

Vicki Allan, Ph.D. Major Professor	Vladimir Kulyukin, Ph.D. Committee Member
Curtis Dyreson, Ph.D. Committee Member	David F. Feldon, Ph.D. Vice Provost of Graduate Studies

Utah State University
Logan, Utah

2025

ABSTRACT

Multi-agent Robotaxi Dispatch Coordination in a Real-world Simulation – Optimizing Rider Assignment, Rebalancing, and Charging Using Battery-dependent Rewards and Welfare Maximization

by

Paden Thompson, Master of Science

Utah State University, 2025

Major Professor: Vicki Allan, Ph.D.
Department: Computer Science

We propose a Multi-agent Reinforcement Learning (MARL) approach to coordinate the dispatch of a robotaxi fleet for an Autonomous Mobility-on-Demand (AMoD) service. Our approach trains a modified Deep-Q Learning neural network using rewards that consider the individual battery level of the robotaxis. Each robotaxi acts as an individual agent in a simulated environment and uses the trained neural network to estimate how valuable different trip servicing, repositioning, and charging actions are to them.

Consider multiple robotaxis interacting in the same area; for example, a group of robotaxis needs to decide which nearby rider would be most valuable for each vehicle to pick up. Our approach uses bipartite graph matching to maximize overall system welfare rather than any individual robotaxi's reward.

Charging autonomous robotaxis is often more difficult than charging standard electric vehicles (EVs), as they do not have drivers who can plug them in at publicly available EV chargers. Because of this, our work focuses on a scenario in which robotaxis must return to a central Operations Depot to charge. We use a battery-dependent reward multiplier during Reinforcement Learning training to incentivize robotaxis with a higher battery level to pick up riders and reposition farther away from the Operations Depot.

We develop a real-world simulation (RiderSim) that utilizes a dataset of all 2024 taxi trips reported to the City of Chicago to simulate realistic timing for trip requests around the city. Furthermore, real-life data is used to estimate the time and distance it takes for robotaxis to reposition between community areas. RiderSim is used to train and test our robotaxi dispatch coordination approaches.

Our results show that using our Welfare Maximization matching procedure to coordinate robotaxi action selection significantly increases the service rate and earning potential for an AMoD service. We also show that using our battery-dependent reward multiplier during Reinforcement Learning training promotes robotaxi circulation and causes the service to perform better when trips are more scarce, compared to a model trained without the reward multiplier and given the same amount of training.

(56 pages)

PUBLIC ABSTRACT

Multi-agent Robotaxi Dispatch Coordination in a Real-world Simulation – Optimizing Rider Assignment, Rebalancing, and Charging Using Battery-dependent Rewards and Welfare Maximization

Paden Thompson

We propose an approach to coordinate a robotaxi fleet for an autonomous ride-hail service. This is a service similar to a traditional ride-hailing service (Uber, Lyft), where customers request a ride and are then picked up in a car and dropped off in a new location; except, driverless vehicles called robotaxis are used to transport the customers.

Our approach teaches helpful coordination strategies to a robotaxi fleet while taking into account the individual battery level of the robotaxis. Each robotaxi acts as an individual agent in our simulation and can choose to pick up a rider, reposition to a new area, or charge. Consider a group of robotaxis that needs to decide which nearby rider would be most valuable for each vehicle to pick up. Our approach maximizes the benefit of the overall system rather than any individual robotaxi's reward.

Charging autonomous robotaxis is often more difficult than charging standard electric vehicles (EVs), as they do not have drivers who can plug them in at publicly available EV chargers. Because of this, our work focuses on a scenario in which robotaxis must return to a central Operations Depot to charge. We incentivize robotaxis with a higher battery level to pick up riders and reposition farther away from the Operations Depot.

We develop a real-world simulation that utilizes data from the City of Chicago to simulate realistic timing for trip requests around the city. Our results show that using our learning strategies to coordinate robotaxi actions significantly increases the service rate and earning potential for an autonomous ride-hail service.

CONTENTS

Page

LIST OF TABLES

Page

LIST OF FIGURES

Page

Chapter 1

Introduction

As self-driving technology matures, Autonomous Mobility-as-a-Service is emerging as a promising technology for urban transportation. Self-driving vehicles, owned and operated as a fleet by an Autonomous Mobility-on-Demand (AMoD) provider, are commonly referred to as robotaxis. Companies like Zoox and Waymo have already deployed robotaxi fleets in populated cities such as San Francisco, Los Angeles, and Phoenix [1].

A robotaxi AMoD service operates similarly to traditional ride-hailing services (such as Lyft and Uber): a customer requests a trip, after which a vehicle picks them up and drives them to their desired destination. However, robotaxi AMoD services have some distinct advantages.

First, driverless vehicles have the potential to be much safer than human drivers in avoiding road collisions [2]. Autonomous driving computer systems do not become distracted the same way human drivers do, can react more quickly to changes in the environment, and can use sensors that see a wider field of view with different senses than humans (e.g., Lidars and thermal cameras [3]).

Second, traditional ride-hailing services utilize contracted drivers with private vehicles who choose their own working hours and location. The ride-hailing platform must ensure some degree of fairness between the individual drivers' opportunities [4]. A robotaxi service does not need to distribute trips fairly between vehicles but can prioritize overall service effectiveness instead. Fleet rebalancing can be coordinated to position robotaxis in the best locations to deal with rider demand.

Typically, robotaxis are battery-powered electric vehicles (EVs) that require charging to operate. However, since these vehicles have no drivers, there is currently no way for robotaxis to utilize public charging stations that driver-operated EVs typically use. Because of this, robotaxi companies must set up operations depots (Ops Depots) to house the employees and infrastructure necessary for charging, cleaning, repairing, and staging robotaxis for service (for example, Zoox's Las Vegas, Nevada headquarters [5]). Furthermore, due to geofencing constraints and limited fleet sizes, robotaxi companies are currently servicing dense urban environments, so they often use a limited number of Ops Depots to stage the robotaxis for an entire city.

Our work sets out to find a solution to the logistics problem created in the most limited case: How can service effectiveness be maximized and costs minimized if robotaxis need to return to a single Ops Depot to charge?

Our work focuses on coordinating robotaxi fleet interactions during three phases of robotaxi dispatch:

- *Rider Assignment:* Which robotaxi should pick up a given rider?
- *Fleet Rebalancing:* Where should robotaxis (without active riders) be repositioned?
- *Charging:* Which robotaxis should be charging?

By modeling an AMoD service as a multi-agent system, where each robotaxi is a separate agent with decision-making processes, reinforcement learning (RL) can uncover strategies that improve interaction behavior.

1.1 Proposed Dispatch Approach

We introduce an AMoD dispatch approach that trains a modified Deep-Q Learning neural network (NN) to learn the value of a given action for a robotaxi within a simulated environment.

To train the NN, we use a simulation called RiderSim, which uses real-world data from the City of Chicago. RiderSim runs a timestep-based simulation that keeps track of the battery level and location for a fleet of robotaxis and advertises trip requests that correspond to real taxi trips serviced in Chicago in 2024.

We use a battery-dependent reward multiplier, called the Battery-level Distance Incentive (BDI), during training to incentivize robotaxi circulation. So, when their battery level is high, they prefer actions farther away from the Ops Depot, and when their battery level is low, they prefer actions nearby the Ops Depot. We compare training a NN with and without the BDI reward multiplier.

When multiple robotaxis are in a position to perform the same limited action (e.g., pick up a specific rider), we propose a Welfare Maximization procedure that matches robotaxi agents with the actions that will maximize overall welfare, rather than any individual robotaxi's reward. We test different dispatch approaches with and without Welfare Maximization to compare against methods that let individual robotaxi agents perform the action they estimate to have the highest reward for themselves, rather than what's best for the overall AMoD service.

1.2 Previous Work

Correa et al. [6] studied scheduling strategies for general EV charging, taking into account variable electricity costs and vehicle demand. But, they focused on charging privately owned vehicles without considering an overall dispatch approach that would be needed for a mobility-on-demand service.

Wang et al. [7] did take into account aspects of a taxi service in their approach to coordinating charging of EVs in a regular taxi fleet (not autonomous). Still, in their scenario, human drivers operate the EV taxis and decide when to request to charge.

Yao et al. [8] investigated various methods to predict taxi trip demand, ranging from using historical data averages to employing a spatial-temporal neural network approach that also considers other external factors, such as weather and holidays, in demand predictions. However, this study did not attempt to simulate a dispatch approach to reposition a fleet of taxis to service the predicted trip demand.

Autonomous Fleet Dispatch

Past research papers on autonomous fleet dispatch algorithms often use simulations similar to the one developed for this work and have also utilized real-life ride-hail and taxi cab data from cities such as New York City [9, 10], Chicago [4], and Chengdu, China [11, 12].

Zhou et al. [9] employed a deep RL dispatch approach for autonomous taxis to service incoming rider requests and estimate the optimal repositioning actions in response to uncertain future rider demand. Qi et al. [10] employed a genetic algorithm to optimize robotaxi dispatch behavior during a pandemic to prevent virus infection rates caused by customer contact while minimizing travel costs and rider wait times. However, neither of these studies considered vehicle battery capacity or charging limitations.

Li [4] investigated RL approaches to rebalance autonomous and non-autonomous ride-hail service fleets. For autonomous fleets, their approach utilized information about the location distribution of robotaxis and customer orders to reposition the robotaxis more effectively, allowing them to pick up as many riders as possible. Their approach utilizes Deep-Q reinforcement learning, where all robotaxi agents share a replay buffer, and unique experiences are prioritized, ensuring that agents are not limited to learning from the same few, most common experiences. The agents were trained to use priority destination sampling assignment, which prioritizes picking up passengers with destinations that are in areas with higher rider demand. In the case of traditional, non-autonomous ride-hailing, Li's approach includes individual driver profit as a relevant metric to entice private drivers to reposition, which is not necessary for an autonomous fleet. In both cases, their approach does not consider the service implications of having to charge the ride-hail fleet.

The authors of [11, 12] also used RL approaches and tried to jointly optimize rider assignment and repositioning, along with charging the robotaxi fleet. Maximum-weight bipartite graph matching is used in [11], similar to our work, but it is applied only for repositioning assignment, excluding trip allocation.

The simulations in both [11, 12] simplify the robotaxi dispatch scenario by defining city regions using uniform hexagons, regardless of street layout, and assuming robotaxis can charge at any number of publicly available EV chargers across the entire city. In contrast, our work utilizes city regions defined by the City of Chicago community area boundaries [13] (see Figure 1.1) and employs real-world data to estimate travel times between areas [14]. We also simulate a more likely scenario where the robotaxi fleet has a set Ops Depot [5] where all robotaxis must return to charge, since they cannot use regular EV charges.

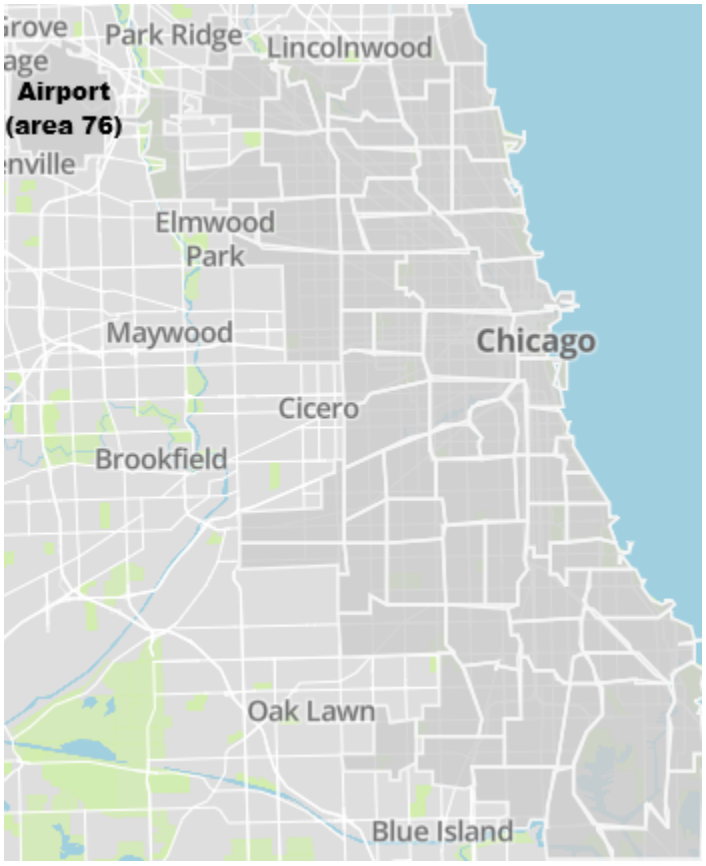

Figure 1.1: Full map of the Chicago community area boundaries (dark gray areas are in the City of Chicago). For example, the community area at the top left corner of the map is the O'Hare International Airport (area 76).

Chapter 2

Methodology

We use a Multi-agent Reinforcement Learning (MARL) approach to simulate the dispatch operations of an AMoD service.

Our simulated environment consists of a fleet of robotaxi vehicles (V) and incoming trip requests (Q). Trip requests are advertised at the start of each discrete simulation timestep and occur when a customer of the AMoD service (referred to as a rider) requests to be picked up in a specific area and transported to a destination area (trip request).

At each timestep, a robotaxi agent has a state consisting of its location and battery level. Actions taken at each timestep can deplete or recharge a robotaxi's battery, depending on the action type. The location of a robotaxi is denoted by an area ID number, which corresponds to one of the three phases of robotaxi activity:

- *Area 0*: robotaxi is in service – actively transporting a rider on a trip
- *Area 1 - 77*: robotaxi is located in one of the 77 Chicago areas (as defined by the City of Chicago community area boundaries, see Figure 1.1) – with no active rider
- *Area 78*: robotaxi is charging at the Ops Depot

2.1 Action Choice

For our work, we consider three main action types that a robotaxi can perform:

- **Service a trip request (S)**: a robotaxi picks up a rider and transports them to their destination
- **Reposition (R)**: a robotaxi moves to a new area while not in service (has no active riders) – this includes when a robotaxi "repositions" to the same area it was already in
- **Charge (C)**: a robotaxi recharges its battery at a charging station in the Ops Depot

2.2 Variable Action Set Deep-Q Learning

A modified version of Deep-Q learning that we call Variable Action Set (VAS) Deep-Q learning is utilized to help robotaxi agents learn optimal action strategies. Because the number of trip requests, repositioning options, and charging stations available can change

depending on the time or location of a robotaxi, our learning method needed to allow for a variable-sized action set.

In VAS Deep-Q learning, a NN is trained to output the value of a single action at a time. This is unlike regular Deep-Q learning, where a NN will output multiple values – one corresponding to each action within a statically-sized action set.

To specify which single action the VAS Deep-Q learning NN should estimate a value for, we input action parameters alongside the simulation state information that would be input into a regular Deep-Q NN. With the action type, distance, and time information input into the VAS Deep-Q NN, it can output a single value corresponding to the estimated reward for that specific action. An external process then queries the NN multiple times with different action parameters to compile the values for each possible action a robotaxi could take during a simulation timestep.

Each action type results in a different reward function for the robotaxi agents. During a simulation, all robotaxi agents share a single neural network (NN), which is trained within the MARL environment using action-reward functions.

VAS Deep-Q Neural Network Input

The input vector ($\vec{I}$) for the VAS Deep-Q NN to estimate the value of taking action A at time t for robotaxi v looks like:

$$\vec{I}_t(A,\, v) = [\vec{V}_t,\, \vec{Q}_t,\, \vec{\text{area}}(v),\, \vec{\text{area}}(A_{\text{dest}}),\, d_A,\, t_A,\, \vec{A_{\text{type}}},\, B_v,\, p_v,\, \vec{\text{day}}(t),\, \text{time}(t)\,] \tag{2.1}$$

The first four entries of $\vec{I}$ are vectors with a length of 79 (one integer entry per community area 0 to 78). See Table 2.1 for a description of each input variable for the Deep-Q NN.

2.3 Reward Functions

Reward functions are calculated for each robotaxi agent after an action is completed (the robotaxi will be located in the action's destination area). Each action type has a distinct reward function used during RL training.

The reward functions for Trip Servicing and Repositioning actions can both use our Battery-level Distance Incentive (BDI). The BDI is a multiplier between 0 and 1 used to bias the reward of actions, thereby accelerating the RL training for battery-dependent coordination strategies. The BDI formulation is explained in depth in the next section. We trained two separate Deep-Q NN to compare results: one using the BDI in the reward functions during RL training and the other without using the BDI during training.

2.3.1 Servicing a trip request

The reward (r) for a Trip Servicing action (S) has a high base reward (S_0, set to 25).

$$r_S(A, v) = S_0 \cdot \text{BDI}(A, v) \tag{2.2}$$

A single Trip Servicing action may span multiple timesteps, so this reward is calculated upon completion of the action.

We do not consider the trip fare as part of the reward in this work because we are focused primarily on increasing the overall number of trips served, with dollars earned as a secondary metric. If there was a larger focus on profitability for an AMoD service, then the amount earned for servicing a given trip could be considered in this reward function.

Table 2.1: Input variables for the Deep-Q Neural Network

Variable	Description
$\vec{V_t}$	Distribution of robotaxi vehicles at time S: vector with one entry per area (a) representing the number of robotaxis in that area (V_t^a)
$\vec{Q_t}$	Distribution of Trip Requests at time S: vector with one entry per area (a), representing the number of Trip Requests in that area (Q_t^a)
$\vec{\text{area}}(v)$	One-hot encoding of the location of the robotaxi v: vector with 1 at the area index of the robotaxi's location and 0s for all other area entries
$\vec{\text{area}}(A_{\text{dest}})$	One-hot encoding of the location of action A's destination (A_{dest}): vector with 1 at the area index of the destination and 0s for all other area entries
d_A	Distance it takes to drive from the start of action A to the destination of action A
t_A	Time it takes to perform action A
$\vec{A_{\text{type}}}$	One-hot encoding for the action type: [1,0,0] for Trip Servicing action, [0,1,0] for Repositioning action, and [0,0,1] for Charging action
B_v	Battery level of the robotaxi v
p_v	Battery percentage of the robotaxi v (so robotaxi can know how close the absolute battery level is to maximum capacity of the battery)
$\vec{\text{day}}(t)$	One-hot encoding of the current day of the week: index 0 for Monday through index 6 for Sunday
$\text{time}(t)$	Current time of day: normalized to a value between 0 and 1, where 12:00:00 AM is set as 0.0

For this initial work, we consider each trip to have the same value to encourage business for the theoretical AMoD service. Furthermore, focusing on increasing the number of trips serviced can also be an economically viable goal because it increases riders' trust in an AMoD service, which can lead to increased future demand and customer retention.

2.3.2 Repositioning

The reward for a Repositioning action (R) is similar to the trip servicing reward – except it has a much lower base reward (R_0, set to 1) because we want to incentivize robotaxis to value servicing a trip, if possible, over repositioning.

$$r_R(A, v) = R_0 \cdot \text{BDI}(A, v) \tag{2.3}$$

For this work, we only consider Repositioning actions that take place over a single timestep. Therefore, robotaxis are limited to repositioning to time-adjacent areas (areas that are within one timestep's travel time of the robotaxi's current area).

2.3.3 Charging

The reward for a Charging action C uses a negative base value (C_0, set to -1) to simulate the cost of charging a robotaxi's battery.

$$r_C(A, v) = C_0 \tag{2.4}$$

2.3.4 Valuing Long-term Rewards

We use a discount factor (γ) set to 0.99 while training the Deep-Q NN. With such a large discount factor, agents learn to value long-term rewards. During training, the estimated value for robotaxi v to perform action A at timestep t, $Est(A_t, v)$, is equal to the actual reward received for performing action A_t plus the estimated value of the action performed in the next timestep (A_{t+1}), multiplied by the discount factor.

$$Est(A_t, v) = r(A_t, v) + \gamma Est(A_{t+1}, t) \tag{2.5}$$

This is where another slight deviation from standard Deep-Q Learning occurs in our modified method: two subsequent actions must be simulated to perform the learning process, rather than just one action and the subsequent simulation state. This is because in the VAS Deep-Q Learning method, rewards depend on the action type.

With the large discount factor, robotaxi agents learned to charge their batteries to allow them to service more trips, even though charging incurs a slight negative reward. Furthermore, if the robotaxi agents let their battery level fall below zero, the reward for a Trip Servicing or Repositioning action is set to B_v (which would be a negative value). In other words, the farther they go without charging an empty battery, the more powerful a negative reward is applied.

2.4 Battery-level Distance Incentive

We propose the Battery-level Distance Incentive (BDI) to aid in the RL training phase, enabling our Deep-Q NN to learn battery-level related strategies more efficiently.

The BDI was inspired by convection currents inside lava lamps. In a lava lamp, warm wax blobs initially rise away from the lamp's heat source; however, as the wax blobs cool, they tend to fall back down closer to the heat source, where they can be reheated restart the process. Our approach similarly incentivizes robotaxis with higher battery levels ("warmer" robotaxis) to service trips and reposition farther away from the Ops Depot. But, as their battery level decreases ("cools down"), the robotaxis become increasingly incentivized to service trips and reposition nearer to the Ops Depot, ensuring they are already in position to charge when needed.

The BDI is designed to speed up training in the RL robotaxi agents by providing immediate reward feedback for the following three behaviors:

(1) Drastically reduce the reward when a robotaxi strays too far from the Ops Depot, when they are in danger of running out of battery before being able to return.

(2) Increase the reward for traveling farther away from the Ops Depot when a robotaxi has a high battery level (robotaxis with more battery remaining are better equipped to transport riders from one side of the city to the other).

(3) Increase the reward for returning to the Ops Depot as the robotaxi's battery gets closer to being empty.

When the BDI multiplier is enabled during RL training, an action's reward will be multiplied by the BDI (which produces a value from 0 to 1). If robotaxi v receives the max BDI reward multiplier (1.0), the reward of action A will remain at full value (multiplied by 100%). When the BDI returns a smaller value, the reward value of action A will be proportionally reduced (e.g., if BDI$(A, v) = 0.4$, then the reward of action A would be reduced to 40% of its original value).

We tested RL training with and without the BDI reward multiplier to determine if adding human-curated, battery-dependent behaviors improves robotaxi service metrics.

BDI Formulation

The BDI formulation depends on the battery level of robotaxi v (B_v) after taking action A and how much battery would be used to get from the destination of action A (A_{dest}) back to the Ops Depot (O) – expressed as $B(A_{dest}, O)$.

For simplicity, we track battery level in units of miles, which indicates approximately how many miles a robotaxi can travel with a given amount of battery charge under nominal driving conditions.

The function cushion(A, v) is used to describe what percentage of robotaxi v's battery would remain if it drove directly back to the Ops Depot from the destination of action A:

$$\text{cushion}(A, v) = 1 - \frac{B(A_{dest}, O)}{B_v} \tag{2.6}$$

This value indicates how close the robotaxi v is to its maximum allowable travel radius, which is the point at which there would be insufficient battery to return to the Ops Depot (happens at the point $B(A_{dest}, O) = B_v$).

To complete the BDI formulation, we also use the battery percentage of robotaxi v (p_v) – defined as the robotaxi's current battery level (B_v) divided by its maximum battery capacity (B_{max}):

$$p_v = \frac{B_v}{B_{\text{max}}} \tag{2.7}$$

To avoid extreme incentive behaviors at very high and very low battery levels, in the BDI formulation, we use a variant of p_v referred to as $\hat{p}_v$, which is equal to the value of p_v constrained between 25% and 75%:

$$\hat{p}_v = \begin{cases} 0.25 & 0.0 \leq p_v \leq 0.25 \\ p_v & 0.25 < p_v < 0.75 \\ 0.75 & 0.75 \leq p_v \leq 1.0 \end{cases} \tag{2.8}$$

With all these inputs, the Battery-level Distance Incentive reward multiplier for a given action A and robotaxi v, BDI(A, v), is calculated by the following equation:

$$\text{BDI}(A, v) = 4\Big(\sqrt{\hat{p}_v \cdot \text{cushion}(A, v)} - \hat{p}_v \cdot \text{cushion}(A, v)\Big) \tag{2.9}$$

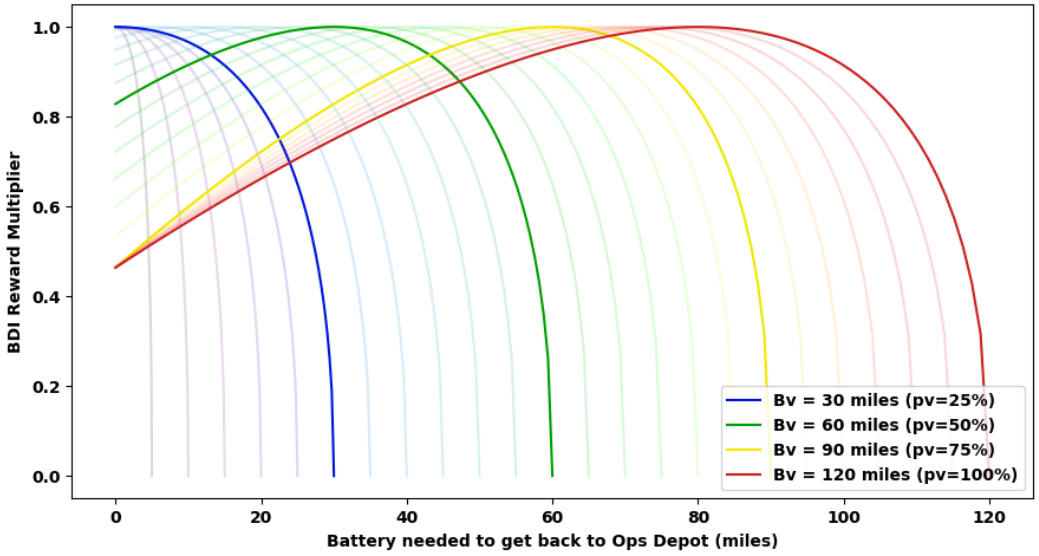

Figure 2.1: Plot showing the Battery-level Distance Incentive reward multiplier for actions with different distances from the Ops Depot, and how the BDI changes for a robotaxi as it loses battery ($B_{\max} = 120$ miles).

Figure 2.1 shows an example of how the BDI function changes as a robotaxi's battery level decreases. Notice how this figure shows the three reward feedback behaviors we wish to incentivize: (1) the BDI multiplier always drops down to 0 when $B(A_{\text{dest}}, O)$ approaches B_v (the maximum allowable travel radius), (2) when the robotaxi's battery level is high, a higher BDI multiplier is given to actions farther away from the Ops Depot, but shifts closer to the Ops Depot as the robotaxi's battery level decreases, (3) the BDI for actions that end up at the Ops Depot are given a low value (0.46) when $p_v \geq 75\%$, but the value slowly shifts up to the maximum multiplier when $p_v \leq 25\%$.

2.5 Welfare Maximization Matching

Consider the case where a rider requests to be picked up in an area with multiple robotaxis: only one robotaxi can actually service the rider's trip. Or, if there is a constraint on the number of charging stations at the Ops Depot, only a limited number of robotaxis can charge at once. Therefore, in our approach, robotaxi agents do not always get to perform the action they estimate to be the most valuable.

To optimally assign actions with limited capacity in each timestep, we use a method akin to solving the min-cost max flow Problem, called maximum-weight bipartite graph matching. A bipartite graph is constructed, where each node represents either a robotaxi or an action, and where each edge represents the ability for a given robotaxi to perform a given action. The edges are weighted by using some action valuation method (for example, a trained Variable Action Set Deep-Q NN that can estimate a single action's reward).

Our Welfare Maximization matching procedure then assigns actions such that the sum of all estimated action values is maximized. We use the LAPJVsp algorithm (Linear assignment problem, Jonker-Volgenant, sparse) [15] to perform the matching.

Figure 2.2 has a diagram that shows the idea behind weighting edges between robotaxis and possible actions to find the most optimal action distribution.

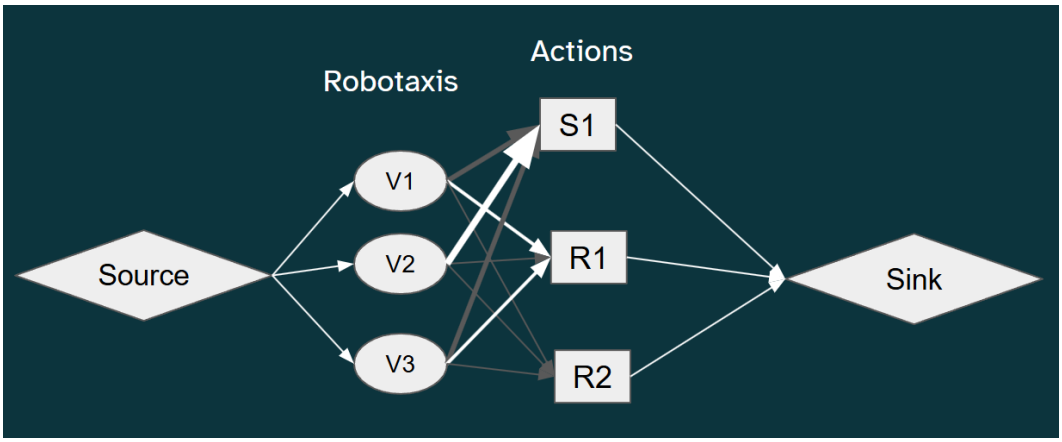

Figure 2.2: Diagram showing the min-cost max flow idea behind the Welfare Maximization matching procedure. The width of an arrow between a robotaxi and an Action node signifies the estimated value for that robotaxi performing the given action. The reward for the overall system is maximized rather than any individual robotaxi's reward. For example, the most preferred action for robotaxi v_1 was to service the trip S_1; however, it was better for the overall welfare of the system to have v_2 service the trip and for v_1 to perform repositioning action R_1 instead.

If enabled during robotaxi action selection, the Welfare Maximization matching procedure takes place each timestep in two phases:

2.5.1 Local Phase

In Local Phase matching, a bipartite graph in matrix form is generated for each area. Each row in the matrix represents a robotaxi in the given area, and each column represents an action that is possible to perform from the given area. The value placed in a given row and column is the estimated value for the column's action according to the row's robotaxi.

Then, maximum weight bipartite graph matching is performed on each area's matrix, and a resulting action assignment matrix is formed. The assignment matrix will place a 1 for each robotaxi row in the action column that it was assigned to perform. The goal of the matching algorithm is to match every row (robotaxi) with a single unique column (action) so that the sum of all the assigned action values is maximized. The matching must be one-to-one, and each robotaxi row must be matched with an action column. However, not all action columns need to be utilized in the match.

Each trip servicing action only corresponds with a single column in the matrix, because only one robotaxi can be assigned to each trip (it is a limited capacity action). However, each repositioning action can have multiple corresponding ones, which signifies that multiple robotaxis can be matched with the same repositioning action. A robotaxi picking a specific area to reposition to does not prevent other robotaxis from also choosing to reposition to the same area. For areas that are time-adjacent to the Ops Depot, charging actions are also added to the bipartite graph matrix, with the number of columns added equal to the number of charging stations remaining.

See Table 2.2 for an example of a bipartite graph matrix that could be generated during Local Phase matching.

In this example, there are two possible trip servicing actions, one possible repositioning action, and one available charging station accessible from the given area. Table 2.3 shows

the resulting action assignment matrix (v_1 would be assigned the S_2 action, v_2 would be assigned the C_1 action, and v_3 would be assigned the S_1 action).

Notice that robotaxis can have widely varied values for the same action. This is because the Deep-Q NN that provides action values can change its output depending on the individual robotaxi's battery level. In this example, v_2 likely has a very low battery level, meaning that it would be very valuable to charge. However, v_3 likelyu has a higher battery level, where servicing a trip is the most valuable action, and charging is

Table 2.2: Local Phase bipartite graph matrix, example 1

	S_1	S_2	R_1	R_1	R_1	C_1
v_1	32.8	**17.5**	1.1	1.1	1.1	10.2
v_2	3.7	3.2	0.8	0.8	0.8	**57.8**
v_3	**42.1**	25.2	1.2	1.2	1.2	1.3

Table 2.3: Local Phase resulting action assignment matrix, example 1

	S_1	S_2	R_1	R_1	R_1	C_1
v_1	0	***1***	0	0	0	0
v_2	0	0	0	0	0	***1***
v_3	***1***	0	0	0	0	0

Notice that multiple columns correspond with a single repositioning action, R_1, which represents to the matching algorithm that multiple robotaxis can select that action at the same time. Table 2.4 shows the same scenario as above, but with different action values. All robotaxis were assigned to the same Repositioning action and Table 2.5 shows the resulting action assignment matrix (each vehicle is assigned to a unique column corresponding to the R_1 action).

Table 2.4: Local Phase bipartite graph matrix, example 2

	S_1	S_2	R_1	R_1	R_1	C_1
v_1	2.5	5.1	**11.1**	11.1	11.1	10.2
v_2	12.4	5.8	19.6	**19.6**	19.6	13.7
v_3	2.6	5.2	21.2	21.2	**21.2**	1.3

Table 2.5: Local Phase resulting action assignment matrix, example 2

	S_1	S_2	R_1	R_1	R_1	C_1
v_1	0	0	***1***	0	0	0
v_2	0	0	0	***1***	0	0
v_3	0	0	0	0	***1***	0

robotaxis that receive a trip servicing action in this initial Local Phase matching are locked in and do not participate in the next matching phase; similarly, the corresponding trip servicing action will be removed from the next matching phase.

2.5.2 Adjacent Phase

After the Local Phase matching, there may still be some remaining trip requests that robotaxis within the same area could not service. So, the next matching phase includes robotaxis in time-adjacent areas (areas that are within one timestep's travel time from a given area). robotaxis that were assigned Repositioning or Charging actions in the Local Phase are entered into the Adjacent Phase matching, where they will estimate the reward for leftover trip requests.

A new bipartite graph matrix is generated for each area, which includes action values for the robotaxis from time-adjacent areas. This matrix adds a column for each remaining trip servicing action, a column for each remaining charging station (when applicable), and finally, one column for each robotaxi's previously chosen action from the Local Phase.

See Table 2.6 for an example of a bipartite graph matrix that could be generated during Adjacent Phase matching. In this example, there are two leftover trip servicing actions in the given area, and three time-adjacent robotaxis that were not previously assigned a trip servicing action.

Notice the last three columns, which represent the actions assigned to the three time-adjacent robotaxis in Local Phase matching. In each of these columns, only the vehicle that corresponds to the action enters its estimated value. The other vehicles do not enter a value for another robotaxi's previously assigned actions, which indicates to the matching algorithm that it is not possible for them to perform that action. Table 2.7 shows the resulting action assignment matrix for this example (v_1 would be assigned to its previously selected Repositioning action R_{v_1} from the local matching phase, v_5 would be assigned the S_7 action, and v_6 would be assigned the S_4 action).

Once again, values for the same trip servicing action can differ from robotaxi to robotaxi because of their individual battery level. In this example, it is likely that v_5 has a medium battery level, where within its own local area, the best action was to charge. However, now that it found a trip in a time-adjacent area, it is better to service the trip than charge at the given moment.

Table 2.6: Adjacent Phase bipartite graph matrix, example

	S_4	S_7	R_{v_1}	C_{v_5}	R_{v_6}
v_1	17.8	18.5	**20.9**	–	–
v_5	7.5	**17.9**	–	12.7	–
v_6	**45.6**	51.2	–	–	2.4

Table 2.7: Adjacent Phase resulting action assignment matrix, example

	S_4	S_7	R_{v_1}	C_{v_5}	R_{v_6}
v_1	0	0	***1***	0	0
v_5	0	***1***	0	0	0
v_6	***1***	0	0	0	0

Maximum weight matching is then performed on each area's adjacent matching bipartite graph matrix. If a robotaxi is assigned a trip servicing action in a given area, its action is locked in, and it will not be included in subsequent matrices. robotaxis that get assigned repositioning or charging actions can continue to vie for trips in other time-adjacent areas until all areas are iterated through, after which all actions are locked in.

Because the Adjacent Phase matching can be affected by the order in which each area is considered, a different random area ordering is used for each timestep.

2.5.3 Reason for Two Matching Phases

There are a few reasons the Welfare Maximization matching procedure is split into two separate phases. First, larger bipartite graph matrices are more expensive to match and more difficult to construct. The matrices could grow considerably if time-adjacent robotaxis were considered in the first matching phase. By locking in many robotaxi and trip servicing actions during the Local Phase matching, it is much less computationally intensive to consider the remaining time-adjacent robotaxis and actions in the Adjacent Phase.

Second, to implement a Welfare Maximization system in real life, the dispatch system would have to start with an initial radius to search for robotaxis to assign to nearby trips. Then, if no robotaxi was available in this initial radius, the search radius would likely be expanded. Selecting a nearby robotaxi first is usually beneficial, since being closer to the rider would shorten the pickup time and allow the robotaxi to use less battery power to get there.

Chapter 3

Real-world Simulation

A simulation called RiderSim was created to train, validate, and test our proposed robotaxi dispatch coordination approaches.

3.1 Dataset

RiderSim utilizes a real-world dataset to simulate trips being requested throughout the day. This dataset includes all reported taxi trips to the City of Chicago in 2024 [14]. The simulation uses the following fields from the 2024 Chicago Taxi Trips dataset to simulate a rider request:

- Pickup time (date/time)
- Pickup community area
- Trip distance (miles)
- Trip duration (seconds)
- Dropoff community area
- Total trip fare ($)

An auxiliary dataset from the City of Chicago maps the community area numbers with their corresponding geographic boundaries and common neighborhood names [13, 16].

Data Cleaning

Some data entries within the Chicago taxi trips dataset are outliers, most likely due to recording errors. For example, there are data entries where a taxi trip took over 24 hours to complete (probably caused by a typo when recording the date) or the trip distance was hundreds of miles. For our simulation, we removed the data entries where the trip time lasted over 2 hours (7200 seconds) and entries where the trip distance was over 50 miles (approximately the maximum possible distance for a trip from one corner of Chicago to the other).

3.2 Running the Simulation

RiderSim provides the environment and reward feedback for our multi-agent RL system. It uses timesteps corresponding to a 15-minute interval, which matches the resolution for the discrete pickup times from the Chicago Taxi Trips dataset [14] and has been used in other taxi simulation research [11].

At the start of each timestep, RiderSim provides the state for each trip request and robotaxi agent. The following steps are performed to transform the state forward in time to the next timestep:

1. **Field trip requests**: RiderSim advertises trip requests in the different areas. Each trip request corresponds to a trip entry in the Chicago taxi trip dataset (and these trip entries have a pickup time that matches the current simulation timestep). For simplicity, no capability is provided for riders to reserve trips for future timesteps.

2. **Obtain dispatch actions**: RiderSim queries for which action each robotaxi will take in this timestep. In this step, RiderSim provides the Deep-Q NN with the necessary input so that each robotaxi can estimate a value for each potential action (see Table 2.1). The Welfare Maximization matching procedure can be enabled here to coordinate action assignment between robotaxis.

3. **Calculate rewards**: RiderSim uses the previous and currently selected action of a given robotaxi, as well as the current state of the simulated environment, to calculate the actual reward for each robotaxi agent's previous action. When enabled in RL training, the Battery-level Distance Incentive (BDI) reward multiplier is applied during this step. See equations 2.2, 2.3, and 2.4 for the exact reward functions used for each action type, and equation 2.9 for details on the BDI reward multiplier.

4. **Update robotaxi state**: RiderSim updates the state of each robotaxi per their chosen actions. For a Repositioning action, the robotaxi is moved to the corresponding destination area. For a Trip Servicing action, the robotaxi is moved to the designated "in-service" area (area 0) until the trip time is completed (rounded to the nearest 15-minute timestep). When a Trip Servicing action is completed within a given timestep, the robotaxi is moved to the trip's destination area. For a Charging action, the robotaxi is repositioned to or remains within the Ops Depot area (area 78). After completing any action, RiderSim updates each robotaxi's battery level to reflect the results of the completed action.

3.2.1 Timestep Resolution

RiderSim uses a 15-minute timestep resolution to match the Chicago taxi cab dataset; however, in real life, an AMoD service would likely have to perform dispatch actions at a greater frequency. The more granular resolution in RiderSim is used because it does not actually locate robotaxis with an exact GPS location in the City of Chicago. In RiderSim, a robotaxi could be positioned anywhere in a community area's boundaries, so exact travel times cannot be calculated. We use the Chicago taxi trip dataset to provide an estimate of how long it takes on average to get from one area to another. The 15-minute timestep also saves computing time for this initial work.

3.2.2 Valid Actions

RiderSim only allows Repositioning actions that take place over a single timestep. This means robotaxis are only allowed to reposition to time-adjacent areas – surrounding areas

where the average travel time to get there from the current area is one timestep or less (after rounding to the nearest 15-minute timestep). The travel time between two areas is calculated by averaging all trip times between them as recorded in the Chicago dataset. Similarly, the distance traveled during a Repositioning action is calculated by averaging all trip distances between the two areas from the Chicago dataset. Note that repositioning within the same area is not free, since intra-area movement also discharges a robotaxi's battery (according to the average trip time and distance for trips given within that single area in the dataset). This also reflects real life, where it is unlikely in a dense, urban setting that a robotaxi would always be able to find an open parking spot and stay stationary while not in service.

It is left up to the dispatch method to keep track of any longer-term repositioning goals for a robotaxi (for example, if a robotaxi wants to reposition to an area that takes multiple timesteps to get to from its current area).

RiderSim allows a robotaxi from any time-adjacent area to service a trip request. Suppose a robotaxi is not in the same area as the trip it is servicing. In that case, RiderSim first discharges the robotaxi's battery according to the time and distance it took to get to the pickup location before starting the trip.

The robotaxi does not have to take an entire timestep to arrive at the trip's pickup location. For example, suppose a trip's pickup area is 4 minutes away and takes 25 minutes to complete. In that case, the total amount of timesteps it would take for the robotaxi to complete the Trip Servicing action is two timesteps (29 minutes rounded to the nearest 15-minute timestep), rather than three timesteps (one timestep taken to get to the pickup area and two more to complete the 25-minute trip). While collecting statistics, only the 25 minutes actually servicing the trip is considered "trip time".

Similar logic applies for Charging actions, where a robotaxi from any time-adjacent area to the Ops Depot (area 78) can choose a Charging action. However, the time it takes for the robotaxi to arrive at the Ops Depot is deducted from the charging time (and the battery is initially discharged according to the time and distance it took for the robotaxi to arrive).

Running out of Battery

RiderSim will not allow a robotaxi to take a trip servicing or repositioning action with a destination outside the robotaxi's maximum allowable travel radius. In other words, actions that end up too far from the Ops Depot for a robotaxi to return with a positive battery level are not allowed to be selected ($\text{cushion}(A, v) < 0$, see equation 2.6). For Welfare Maximization, this means the action's value will not be input for that robotaxi in the matching matrix.

Suppose there are no possible actions left for a robotaxi after eliminating the actions outside the maximum allowable travel radius. In that case, RiderSim will force the robotaxi to perform a Charging action, or, if charging is not currently possible, perform the trip servicing or repositioning action that will bring the robotaxi closest to the Ops Depot.

3.3 Simulated Operations Depot

Within RiderSim, the Lower West Side neighborhood (community area 31) was chosen to house the simulated robotaxi Ops Depot. This neighborhood serves as a staging area for FedEx Ground delivery trucks in Chicago, featuring numerous industrial warehouses and parking lots, and is still conveniently close to downtown Chicago. These features suggest that the area would be a suitable location for an Ops Depot. An Ops Depot requires ample space to house charging infrastructure and dispatch operations for a large vehicle fleet, preferably in an area where real estate prices are not as high as in the downtown core. See

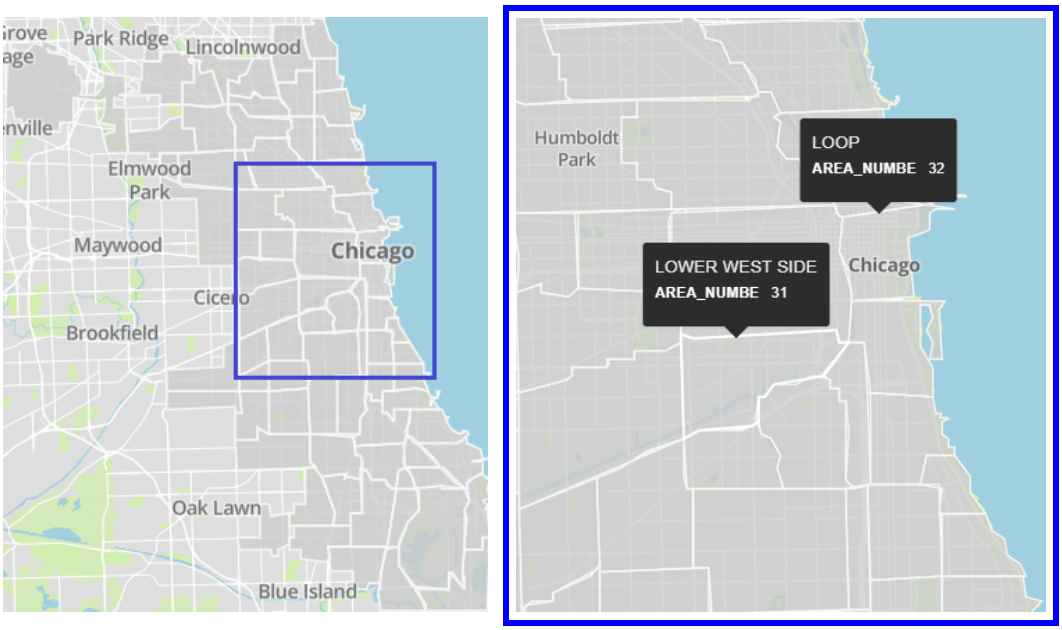

Figure 3.1:
Left: Full map of the Chicago community area boundaries (dark gray areas are in the City of Chicago).
Right: Map zoomed in to show the distance between the Lower West Side neighborhood (area 31), which houses the simulated Ops Depot, and Chicago's main downtown Loop neighborhood (area 32).

Figure 3.1 to see the distance between the simulated Ops Depot in area 31 and downtown Chicago.

To simplify statistics collection when robotaxis are actively charging versus just repositioning within area 31, RiderSim designates a special community area, number 78, for the Ops Depot. RiderSim uses identical average travel distances and travel times for areas 31 and 78.

3.4 Updating Battery Levels

3.4.1 Battery Discharge

EV batteries not only lose charge according to the distance a vehicle travels but also discharge over time, even when not in motion, due to static energy costs such as running the EV's computer system, playing music, or adjusting climate controls [17]. For this reason, the simulated amount of battery a robotaxi discharges ($B_{\text{discharge}}$) while performing a given action (A) depends on the distance it drove to get to the destination of the action (d_A) and the amount of time it took to drive there (t_A). Constant ψ denotes how much charge is lost to static energy costs per unit of time. For our work, we have ψ set to a loss of 5 miles of battery charge per hour of elapsed time.

$$B_{\text{discharge}}(A) = d_A + \psi t_A \tag{3.1}$$

For a robotaxi servicing a trip (S), t_S and d_S are set by the actual time and distance

for the real-life trip reported in the Chicago dataset. However, there is no direct measure for robotaxi repositioning actions (R) between two areas in the Chicago dataset. For this reason, t_R and d_R are set by the average time and distance of all reported trips between the two areas in the Chicago dataset.

3.4.2 Battery Charge

EV batteries do not charge linearly. As an EV battery approaches its maximum capacity, the amount of energy it can accept from a charger decreases, meaning it will charge more slowly [20]. RiderSim approximates this behavior when a robotaxi charges at the Ops Depot.

We denote the maximum rate of charge for a robotaxi as β. When a robotaxi's battery is mostly empty, its actual charge rate per unit of time will be the maximum rate (β). However, to emulate the real-life behavior of battery charging, the actual charge rate will be limited after the battery percentage (p_v) exceeds 50%. It will decrease until the robotaxi's battery level approaches max capacity ($p_v = 1.0$), at which point the actual charge rate will be reduced to 20% of the maximum, β.

The amount of battery charged for robotaxi v over a given amount of time t in RiderSim is expressed as:

$$B_{\text{charge}_v}(t) = \begin{cases} \beta t & p_v < 0.5 \\ (1.8 - 1.6p_v)\beta t & p_v \geq 0.5 \end{cases} \tag{3.2}$$

This equation is just an approximation of real-life behavior, and actual charging curves will vary from EV model to model [18]. See Figure 3.2 for examples of simulated battery charging curves. In this example, β equals 225 miles per hour of charging and the battery's maximum capacity is 120 miles. For an average EV that consumes approximately 0.35 kWh per mile of driving [19], this example simulates the charging curves for an EV with a 42 kWh battery at a 72 kW charger.

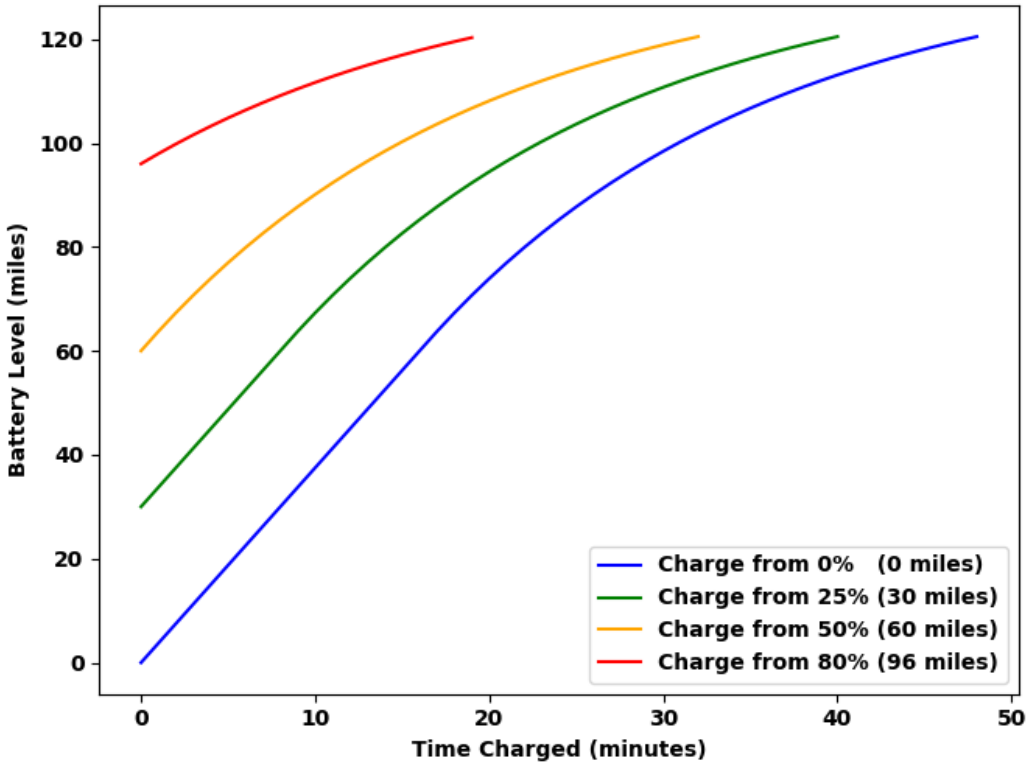

Figure 3.2: Simulated battery charging curves for a robotaxi with maximum battery capacity ($B_{\max}$) equal to 120 miles, starting at different initial battery percentages.

Chapter 4

Results

4.1 Reinforcement Learning Model Training

To provide a side-by-side comparison for the Battery-level Distance Incentive (BDI), we trained two different Variable Action Set (VAS) Deep-Q Neural Networks (NNs). The first NN was trained using reward functions that included the BDI reward multiplier; the other NN was trained using the same base reward functions, except that they excluded the BDI reward multiplier. See equations 2.2, 2.3, and 2.4 for the exact reward functions used during RL training, and equation 2.9 for details on the BDI reward multiplier.

To allow for a fair comparison between the two NNs, both used the same training parameters and started with the same random weights and biases. See Table 4.1 for the specific NN training parameters used.

Table 4.1: Neural Network Training Parameters

Parameter	Value
Layer Sizes	$331 \times 64 \times 64 \times 1$
Activation Functions	ReLu, ReLu, Linear
Discount Factor (γ)	0.99
Exploration Parameter (ϵ)	1.0 - 0.01 (decreased over 10000 training steps)
Learning Rate (η)	0.001
Learning Batch Size	128

Both NNs received the same amount of training time. The NNs were trained for 100 iterations, with each iteration starting on a random month, day, and hour from the Chicago taxi trip dataset. Each training iteration simulated 100 timesteps worth of trip requests (just over a day) and had a robotaxi fleet of 100 vehicles.

Training Results

After training, both NNs could successfully predict which areas would have more trip requests depending on the time of day, and could assign actions so robotaxis could reposition to those areas and service the trips. Figures 4.1, 4.2, and 4.3 show examples of the BDI-trained NN repositioning robotaxis across the different Chicago community areas throughout the day.

Notice that for in simulation late at night (Figure 4.1), there is very low trip demand, so many robotaxis are housing themselves at the Ops Depot (area 78). In RiderSim, it costs

battery power to be out on the street repositioning, and even repositioning within the same area is not free. This means that for the excess robotaxis, the most efficient place to wait during a low-demand period is at the Ops Depot. They are not all actively charging, but are waiting for later in the day when demand spikes to head out with a full charge, ready to pick up riders.

All three figures show that robotaxi distribution generally follows the patterns of trip requests in the different areas; robotaxi concentration is usually highest in the areas with the a significant number of trip requests. This indicates that the VAS Deep-Q NN successfully learned to predict where trip requests are going to come from throughout the day.

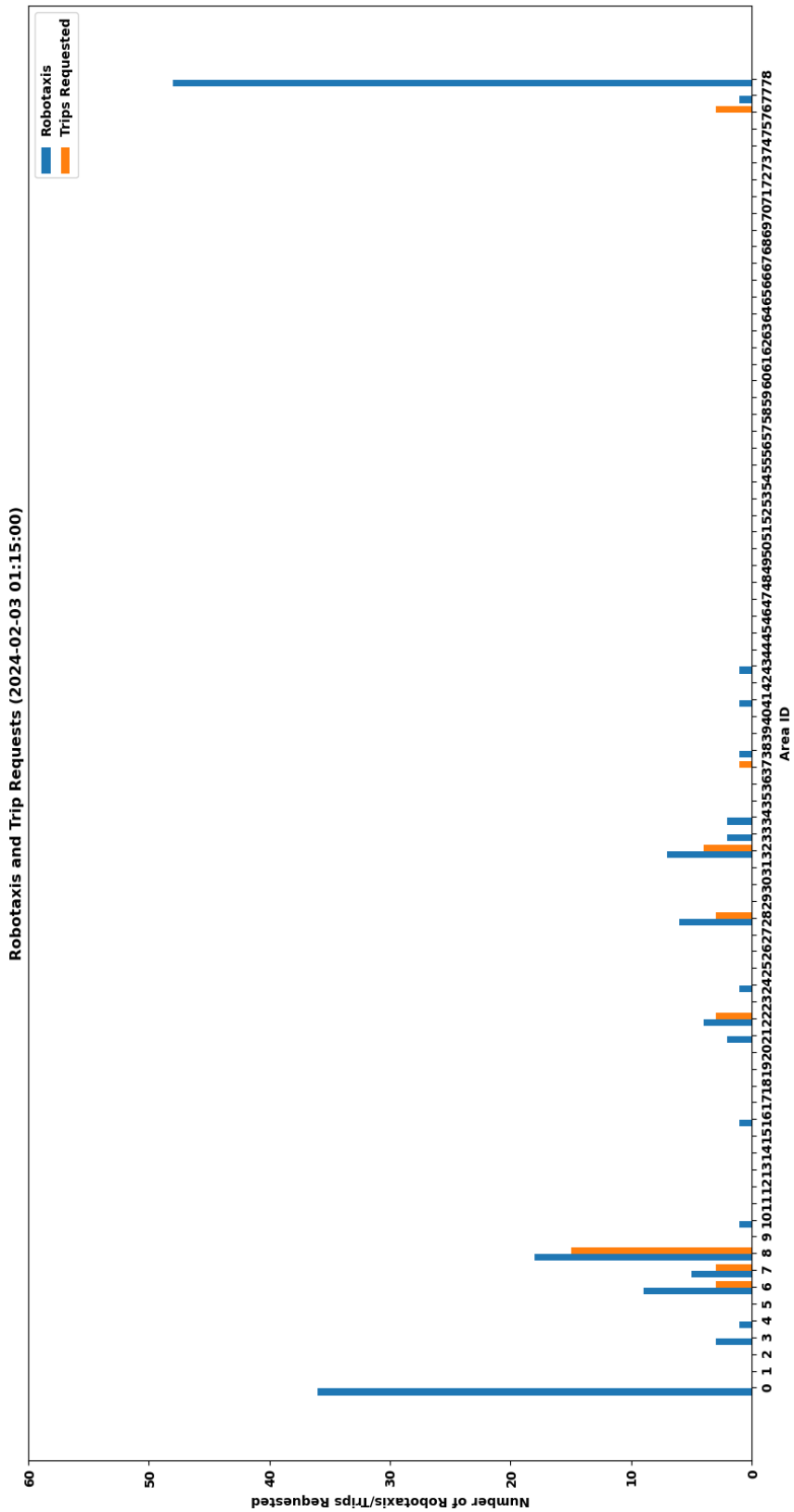

Figure 4.1: Distribution of robotaxis and Trip Requests at night (1:15 AM) – fleet size 150 vehicles. Many robotaxis are waiting at the Ops Depot (area 78) because the NN predicts that not all robotaxis are needed to pick up riders at this time. Therefore, it is more efficient for them to charge or park at the Ops Depot. This reflects the real-life behavior of mobility fleet services, where not all vehicles will be out on the streets ready to pick up passengers during lower-demand periods.

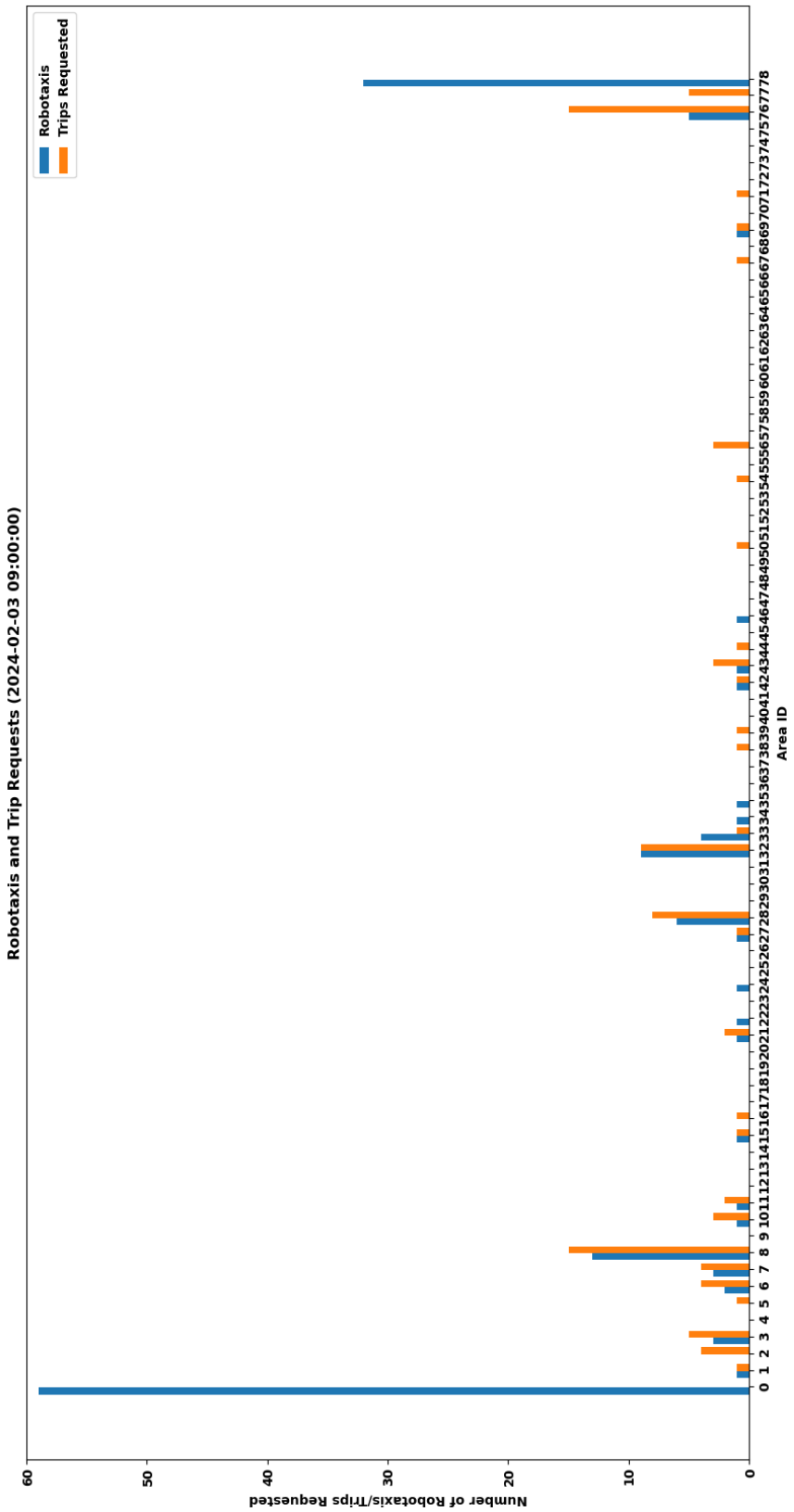

Figure 4.2: Distribution of robotaxis and Trip Requests in the morning (9:00 AM) – fleet size 150 vehicles. Many robotaxis are currently servicing a trip (area 0). Not as many robotaxis are charging at the Ops Depot compared to the night, since there has been an increase in trip demand.

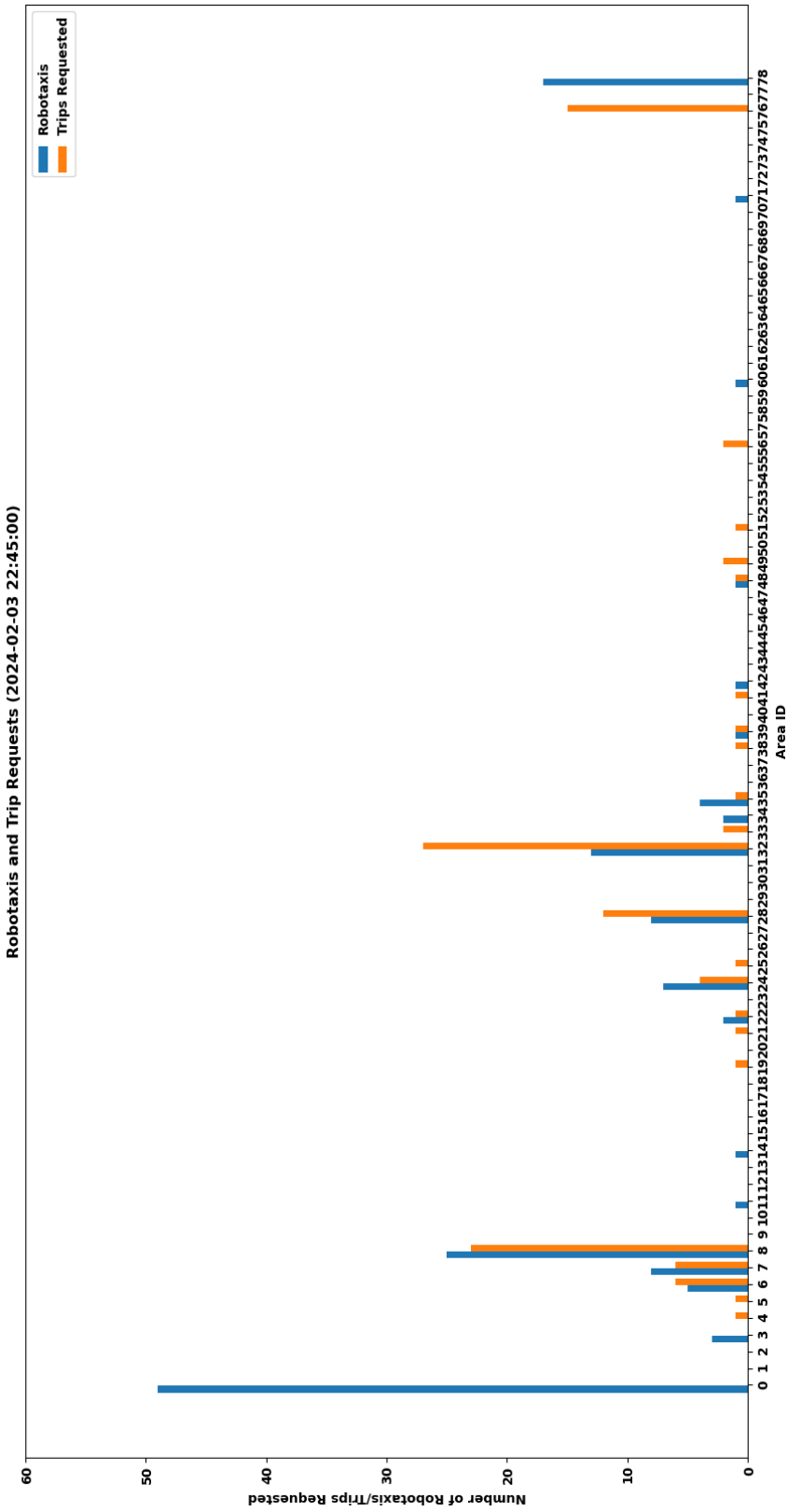

Figure 4.3: Distribution of robotaxis and Trip Requests in the evening (10:45 PM) – fleet size 150 vehicles. The NN continues to predict how many robotaxis are needed in each area to fulfill trip requests, but the fleet is starting to get overwhelmed by the number of trip requests.

4.2 Experimental Setup

To test the performance of our Welfare Maximization matching procedure and Battery-level Distance Incentive (BDI), we simulated six different robotaxi dispatch methods using RiderSim:

- **Basic Approach**: Uses hand-coded value estimates for actions in a hierarchy. The highest valued actions are trips, followed by repositioning actions if no trips are available. When robotaxis run low on battery, they value charging actions the most and prefer repositioning and trip actions that head toward the Ops Depot.
- **Basic Approach + Welfare Maximization**: Same action valuation as Basic Approach, but with the Welfare Maximization matching procedure enabled.
- **No BDI Model**: Uses action valuation from the VAS Deep-Q Neural Network trained without the BDI reward multiplier.
- **No BDI Model + Welfare Maximization**: Same No BDI action valuation as above, but with the Welfare Maximization matching procedure enabled.
- **BDI Model**: Uses action valuation from the VAS Deep-Q Neural Network trained with the BDI reward multiplier.
- **BDI Model + Welfare Maximization**: Same BDI model action valuation as above, but with the Welfare Maximization matching procedure enabled.

When Welfare Maximization was not enabled, robotaxis were ordered from the least battery remaining to the most. Each robotaxi in the sequence was allowed to perform the action it valued most, regardless of other robotaxis' preferences. If a robotaxi earlier in the order chose a trip servicing action, that trip would not be an available option for subsequent robotaxis. This way, robotaxis with the lowest battery remaining had the first pick of charging stations and trips that could help them return to the Ops Depot.

4.2.1 Simulation Parameters

To obtain metrics from RiderSim, each experiment was repeated three times, and the results were averaged. Each dispatch approach was provided with the same three starting states (robotaxi distribution and date) for the three experiment iterations. Each iteration ran for 192 timesteps, which equates to two days of simulated time. The three start dates for the iterations chosen were February 3, 2024, at 12:00 AM, July 3, 2024, at 12:00 AM, and October 3, 2024, at 12:00 AM. These dates were chosen to test three different scenarios in the Chicago dataset: a weekend (Feb. 3-4), a holiday week (July 3-4), and a regular set of weekdays (Oct 3-4). See Table 4.2 for the RiderSim parameters used during the experiment simulations.

Although a 120-mile battery range is quite small for a real-life robotaxi, it is used here to primarily stress the robotaxi dispatch methods and amplify the need to go back to the Ops Depot to charge. Real-life robotaxis likely do not utilize their battery from full to empty while in service, as this can damage an EV battery over time. Therefore, the 120-mile range used in the simulation can be considered the functional range for robotaxi batteries in our experiments.

4.2.2 Performance Metrics

We used the following set of performance metrics to compare how well each dispatch approach performed.

Table 4.2: RiderSim Experiment Simulation Parameters

Parameter	Value
Fleet Size	100, 150, or 200
Timesteps	192 (two simulated days)
Experiment Iterations	3
$B_{\max}$ (maximum robotaxi battery capacity)	120 miles
β (battery charging maximum rate)	225 miles/hour of charging
ψ (battery discharge over time)	5 miles/hour

Service Rate

The Service Rate (SR) is the primary metric investigated in our work. The SR for a given simulation window is defined as the number of trips successfully serviced (S_{success}) divided by the total number of trips requested (S_{total}).

$$\text{SR} = \frac{S_{\text{success}}}{S_{\text{total}}} \tag{4.1}$$

The SR is the best metric for determining how successful a dispatch approach is at capturing ridership around the city. The higher the SR, the more trip requests the robotaxis service can handle. This metric rises when larger robotaxi fleet sizes are simulated (since there are more robotaxis per customer).

Ideal Service Rate

In 2021, it was estimated that Chicago could have up to 1000 taxi cabs operating simultaneously around the city at the same time [21]. Our experiments use smaller fleet sizes of 100, 150, and 200 vehicles, so it would be impossible for the simulated robotaxi fleet to service every trip reported in the Chicago taxi trip dataset.

Trip demand fluctuates significantly throughout the day and can limit the theoretical maximum Service Rate. For example, even though the simulations used in our experiments have an average of 132 trip requests per timestep, the maximum number of requested trips in a single timestep is 336. This means the minimum fleet size that could even theoretically obtain a 100% Service Rate would need at least 336 vehicles.

We propose the Ideal Service Rate to provide a guidepost for analyzing Service Rate results at different fleet sizes. The Ideal Service Rate is defined as the maximum Service Rate possible for a given fleet size, assuming robotaxis are constantly in service (meaning they would never have to charge or reposition).

To calculate the Ideal Service Rate for a given fleet size (n), first, we find how many trips it is possible for the fleet to service in a single timestep. Because trips often take longer than a single timestep, we divide the number of robotaxis in the fleet by the average number of timesteps it takes to complete one trip servicing action ($\text{avg}(t_S)$):

$$\text{MaxTripsPerTimestep}(n) = \frac{n}{\text{avg}(t_S)} \tag{4.2}$$

Next, we find out how many possible trips the fleet could have serviced throughout an entire simulation. To accomplish this, we iterate over the sequence of trip requests from a specific simulation ($\vec{S_{\text{req}}}$). For each timestep from $k = 0$ to the last timestep in the simulation, t_{end}, we sum the maximum number of possible trips that could have been serviced at that timestep, which is either the actual number of trips requested during that

timestep (S^k_{req}), or if that is too large, the maximum trips per timestep for the fleet that we found previously:

$$\text{PossibleTrips}(n, \vec{S_{\text{req}}}) = \sum_{k=0}^{t_{\text{end}}} \min\big(S^k_{\text{req}}, \text{MaxTripsPerTimestep}(n)\big) \tag{4.3}$$

We can then calculate the Ideal Service Rate for a specific fleet and simulation as the number of trips that could possibly have been serviced, divided by the number of trips that were requested in total during the simulation:

$$\text{IdealSR}(n, \vec{S_{\text{req}}}) = \frac{\text{PossibleTrips}(n, \vec{S_{\text{req}}})}{S_{\text{total}}} \tag{4.4}$$

In figures showing Service Rate results, we plot the Ideal Service Rate as a dotted line alongside the actual results to show what is theoretically possible to achieve for that specific experiment.

Battery-use Efficiency

Battery-use Efficiency (BE) is another metric investigated in our work. BE attempts to determine the proportion of battery the robotaxi fleet uses to actually transport riders. BE is defined as the amount of battery discharged for all Trip Servicing actions (B_S) divided by the total amount of battery discharged for all actions (B_{total}).

$$\text{BE} = \frac{B_{\text{S}}}{B_{\text{total}}} \tag{4.5}$$

The BE metric measures how efficiently robotaxis utilize their battery charge to transport riders, rather than constantly repositioning or traveling back to the Ops Depot without any riders. For long enough simulations, the total amount of battery discharged will approximately equal the total amount of battery charged during Charging actions ($B_{\text{total}} \approx B_C$), so this metric can also be seen as a measure of how efficiently the robotaxi fleet is using the electricity used to charge all of the vehicles.

Dollars per Mile

The Dollars per Mile (DPM) metric was also investigated. DPM is defined by the total amount of fare obtained from riders (F_{total}) divided by the total amount of battery discharged for all actions (B_{total}):

$$\text{DPM} = \frac{\text{F}_{\text{total}}}{B_{\text{total}}} \tag{4.6}$$

A higher DPM metric shows the robotaxi service is more profitable per mile of battery charge expended. As mentioned, Service Rate is prioritized over dollars earned for this work, but for a real AMoD service, DPM would be a fundamental metric, so it is included in the results here. DPM is a useful metric to see how efficient a robotaxi service is. It can show that even when a RL approach does not consider the fare price of a trip as part of the action reward, the resulting behavior can have a beneficial impact.

4.3 Experiments

4.3.1 BDI Model versus No BDI Model

The first group of experiments was performed to compare the trained Neural Networks for the BDI training model versus the No BDI training model. Results were simulated for robotaxi fleet sizes of 100, 150, and 200 vehicles. Figures 4.4 and 4.5 show the plotted results for the experiments. The results for the Basic approach were also plotted alongside the other results to show a baseline for what a non-RL approach can provide.

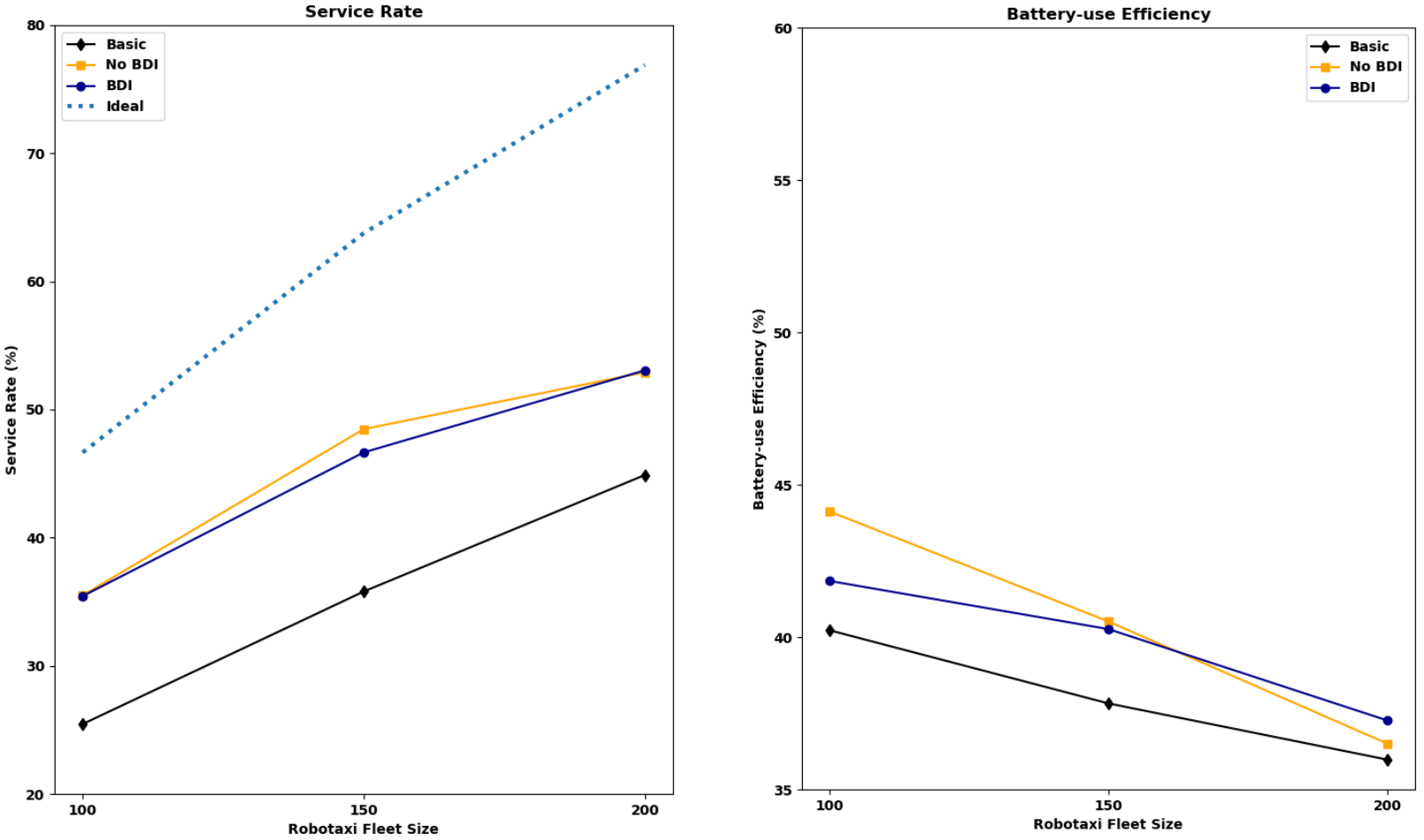

Figure 4.4: Service Rate and Battery-use Efficiency results for varying robotaxi fleet sizes: Basic, No BDI, and BDI dispatch approaches shown.

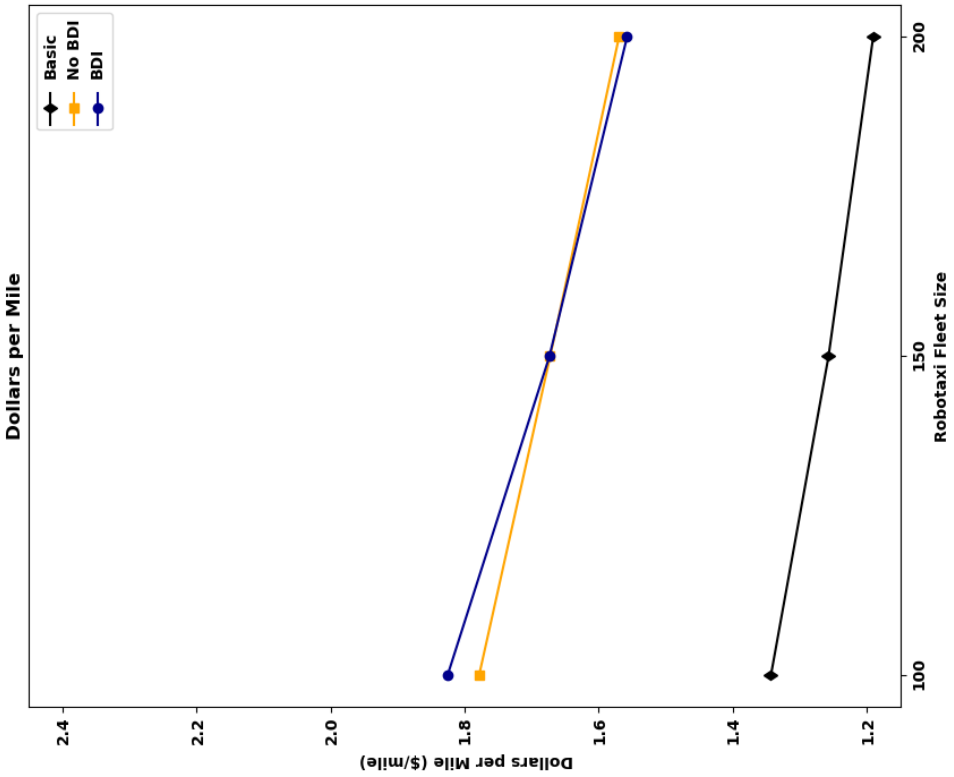

Figure 4.5: Dollars per Mile results for varying robotaxi fleet sizes: Basic, No BDI, and BDI dispatch approaches shown.

Both the BDI and No BDI models show an improvement for each performance metric over the Basic approach. This shows that in nominal conditions, the Deep-Q Learning models were successful.

Given the same amount of training, the BDI and No BDI models ended up with very similar metrics results. This shows that the No BDI NN was able to approximate battery-dependent behaviors, even without the addition of specific battery-dependent reward multipliers. A simpler reward system still lead to promising results.

4.3.2 BDI with Demand Reduction

To show the advantages of the BDI training model, a group of experiments was performed with varying levels of demand reduction.

In RiderSim, demand reduction lowers the number of trip requests used from the Chicago taxi trip dataset. This can be seen as a "Trips Remaining" percent, where 25% would mean that for each timestep in the simulation, only one-fourth of the trip requests were made available to the robotaxis. This effectively decreases the rider demand and makes it more feasible for a smaller robotaxi fleet to get a higher Service Rate; however, it increases competition between robotaxis trying to service the remaining trips.

By reducing demand more and more, we can verify how robust a dispatch method is to increased trip competition.

Figure 4.6 shows the Service Rate results of the BDI and No BDI models under varying levels of demand reduction. Each experiment was run with a robotaxi fleet size of 200 vehicles.

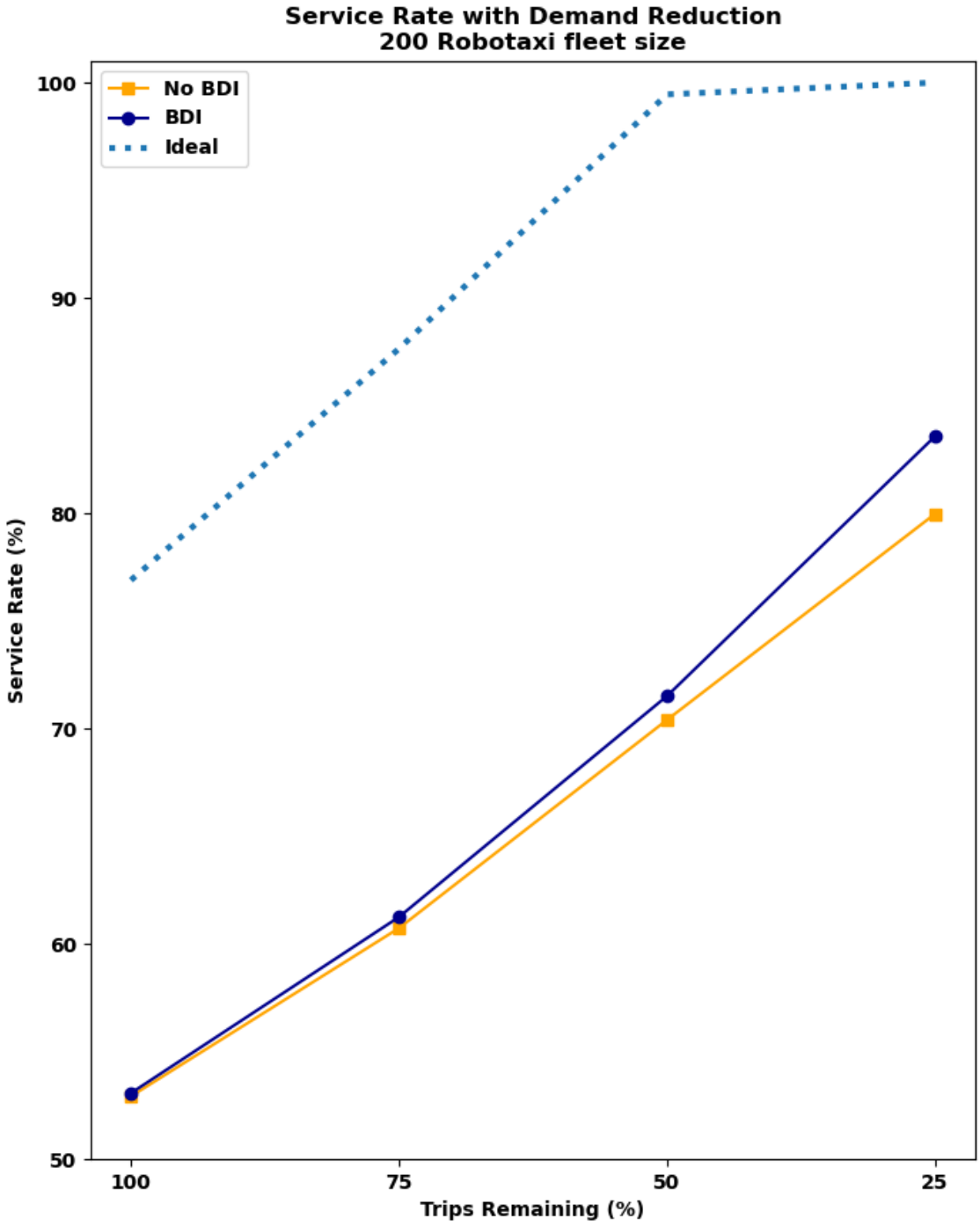

Figure 4.6: Service Rate results for varying trip scarcities (fleet size of 200 robotaxis): No BDI and BDI dispatch approaches shown.

As demand is reduced and fewer trips remain, the BDI model significantly improves its Service Rate over the No BDI model. These results show that when only 25% of trips are used in the simulated experiment, the BDI model can service almost 84% of riders, versus the No BDI model, which can only service around 79%.

This discrepancy is likely because the BDI model promotes more robotaxi circulation by incentivizing robotaxis to accept trip requests and repositioning actions that take them farther away from the Operations Depot when their battery level is high. During training, the No BDI model probably learned that it is easier to service many trips close to the Ops Depot, and not reposition as much. However, when trip are removed during demand reduction, there are fewer and fewer requests near the Ops Depot. At higher trip scarcities, repositioning actions become increasingly valuable because there is no longer an abundance of trip requests to service in one area. The results indicate the BDI model is more proficient at spreading around the robotaxis using repositioning actions than the No BDI Model when fewer trips remain.

A good indicator of how well a dispatch method circulates robotaxis is the number of riders picked up and dropped off at the airport (Chicago community area 76) in a simulation. In Chicago, the O'Hare International Airport is located in the farthest north-west corner of the city (see Figure 1.1) and is the busiest area in terms of trip requests outside the vicinity of Chicago's downtown. A higher number of riders picked up or dropped off at the airport shows that robotaxis consistently travel far away from the Ops Depot.

Notice in Figure 4.7, the BDI model services considerably more trips that have pickups or dropoffs at the airport than the No BDI model. Even before demand reduction, the BDI model services a significant number of airport trips, indicating that it promotes good robotaxi circulation farther from the Ops Depot, even when there is less competition for trips.

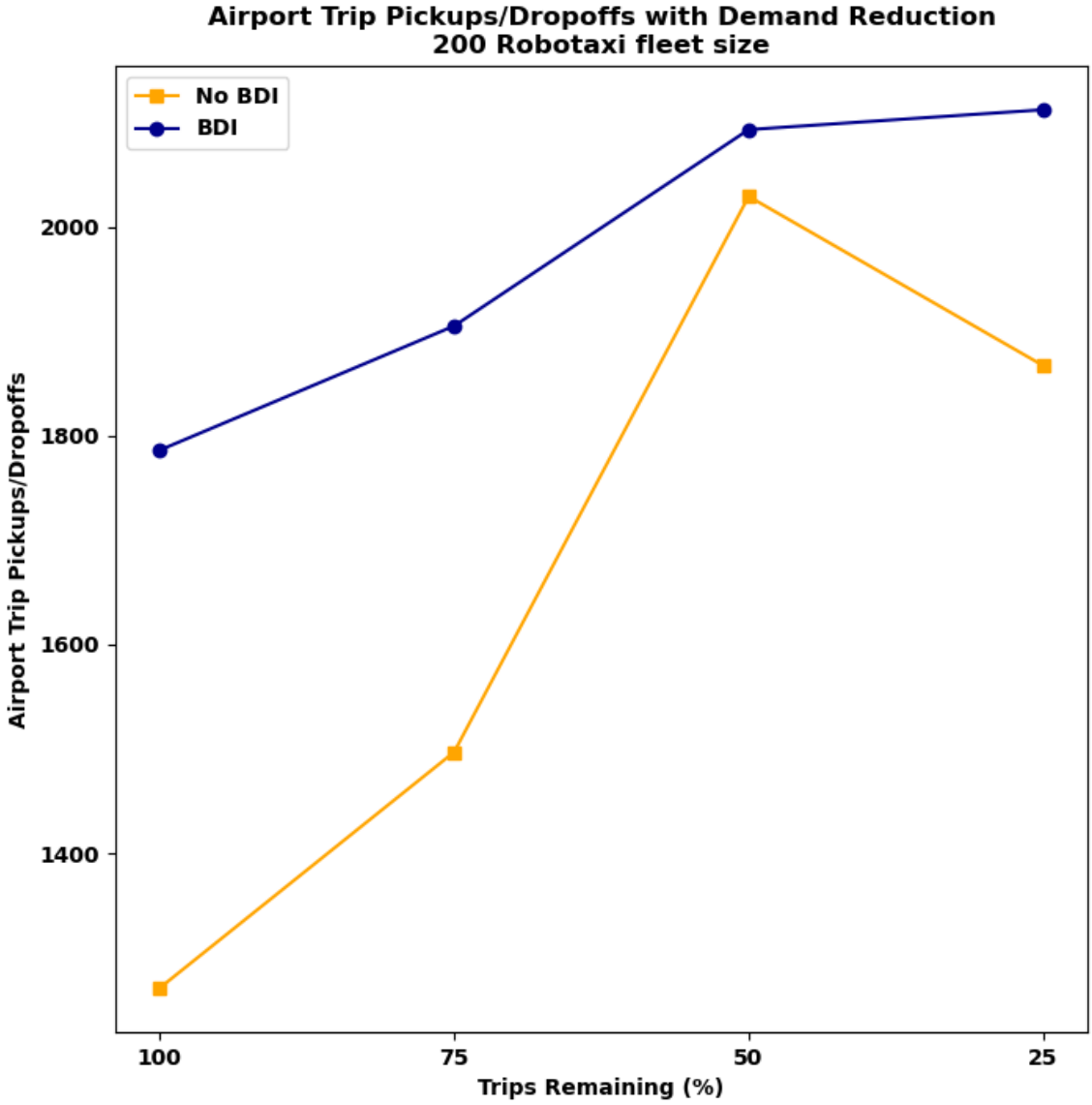

Figure 4.7: Total number of trips serviced with pickups or dropoffs at the O'Hare International Airport (area 76) over the course of six simulated days, for varying trip scarcities (fleet size of 200 robotaxis): No BDI and BDI dispatch approaches shown.

4.3.3 Welfare Maximization

To show the performance of our Welfare Maximization matching procedure, the experiments with 100, 150, and 200 vehicles were run with varying dispatch models with Welfare Maximization enabled or disabled during RiderSim action selection.

Basic approach

Figures 4.8 and 4.9 show the comparison between the Basic approach with and without Welfare Maximization enabled.

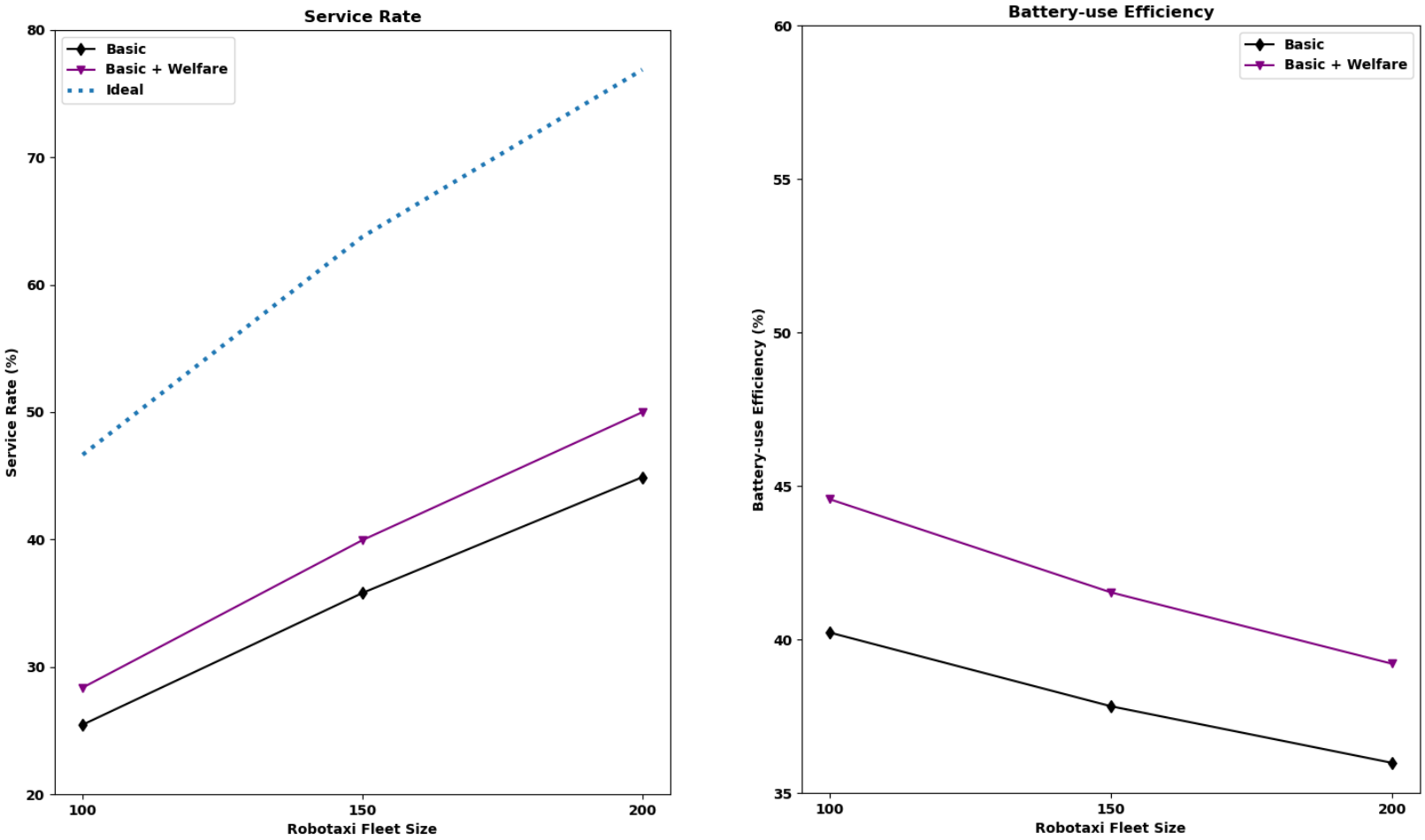

Figure 4.8: Service Rate and Battery-use Efficency results for varying robotaxi fleet size: Basic approach with and without Welfare Maximization shown.

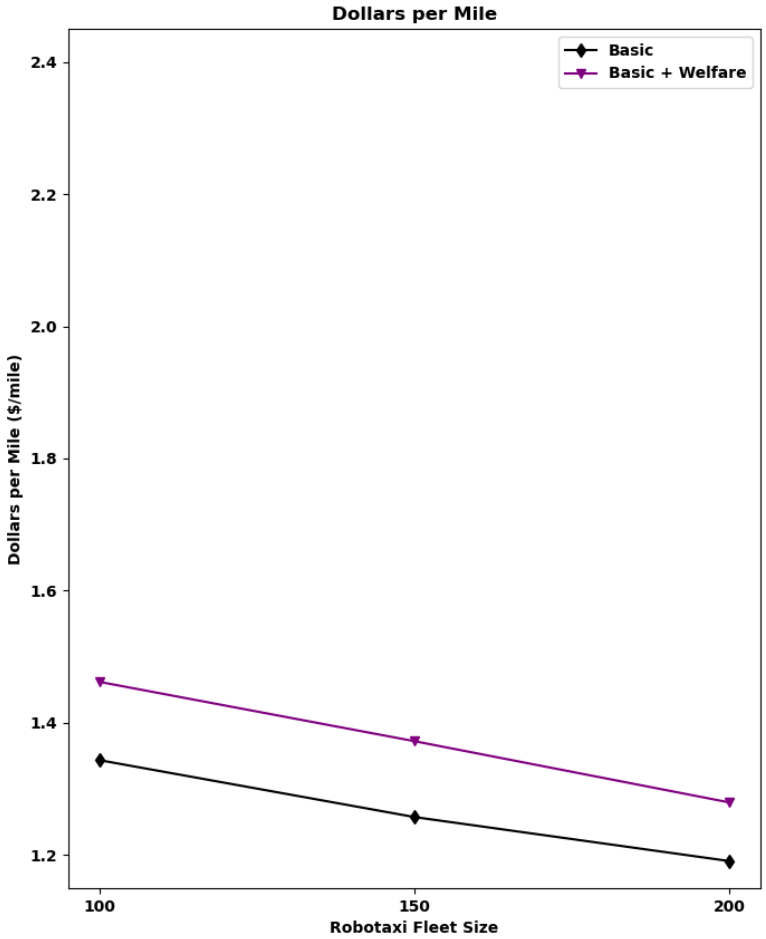

Figure 4.9: Dollars per Mile results for varying robotaxi fleet size: Basic approach with and without Welfare Maximization shown.

Although the Basic approach has the lowest performance metrics of all the methods we tested, these results show that enabling Welfare Maximization can improve all metrics for the Basic approach. The Service Rate goes up by about 5%, the Battery-use Efficiency up by about 4%, and the Dollars per Mile up by about $0.10 per mile across all fleet sizes.

This shows the distinct advantage of an *autonomous* mobility service, versus a regular taxi service with a collection of individual human drivers. Even with a less effective dispatch method, autonomous robotaxis can use fleet coordination to improve overall service behavior.

BDI Model

Figures 4.10 and 4.11 show the comparison between the BDI model with and without Welfare Maximization enabled.

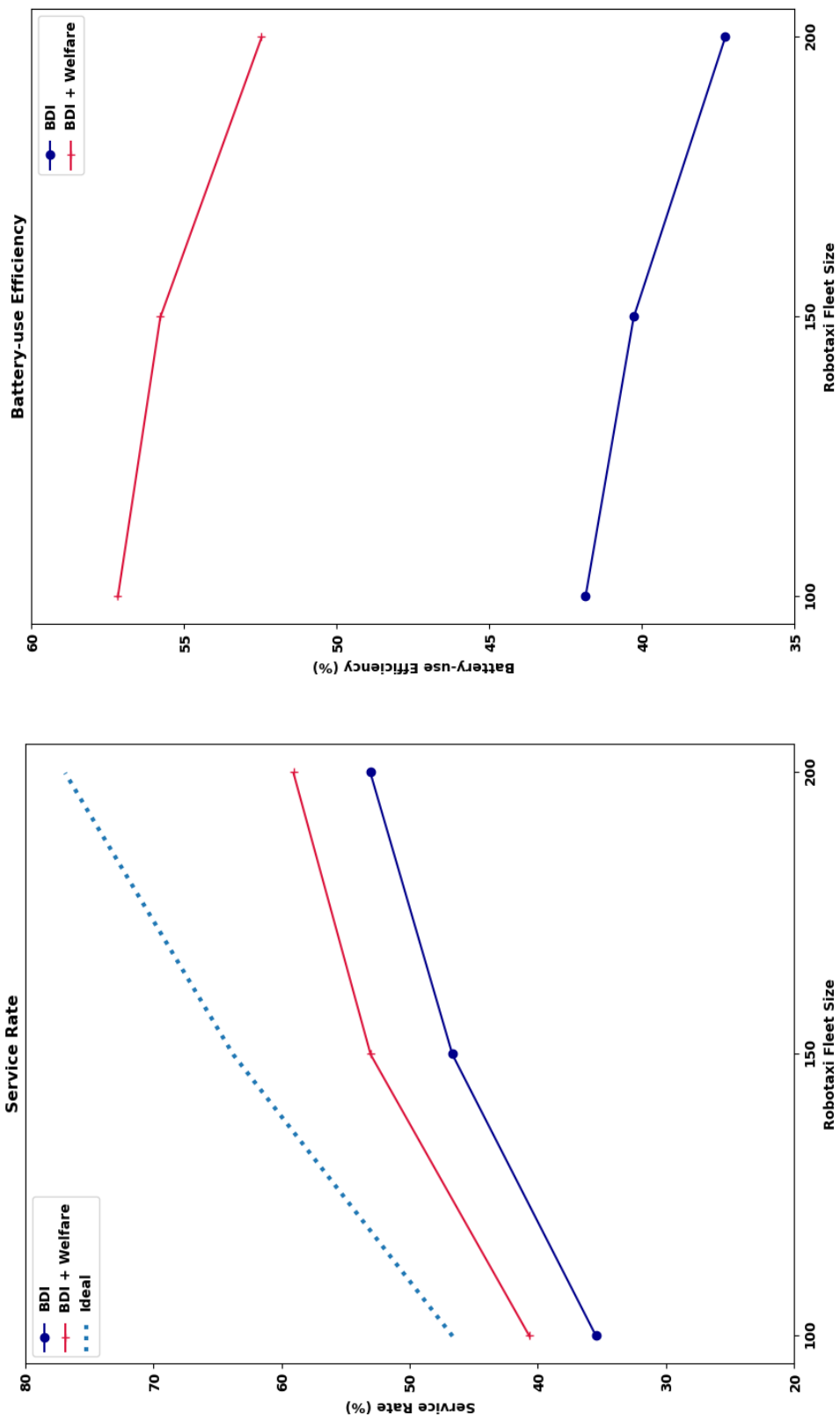

Figure 4.10: Service Rate and Battery-use Efficiency results for varying robotaxi fleet size: BDI model with and without Welfare Maximization shown.

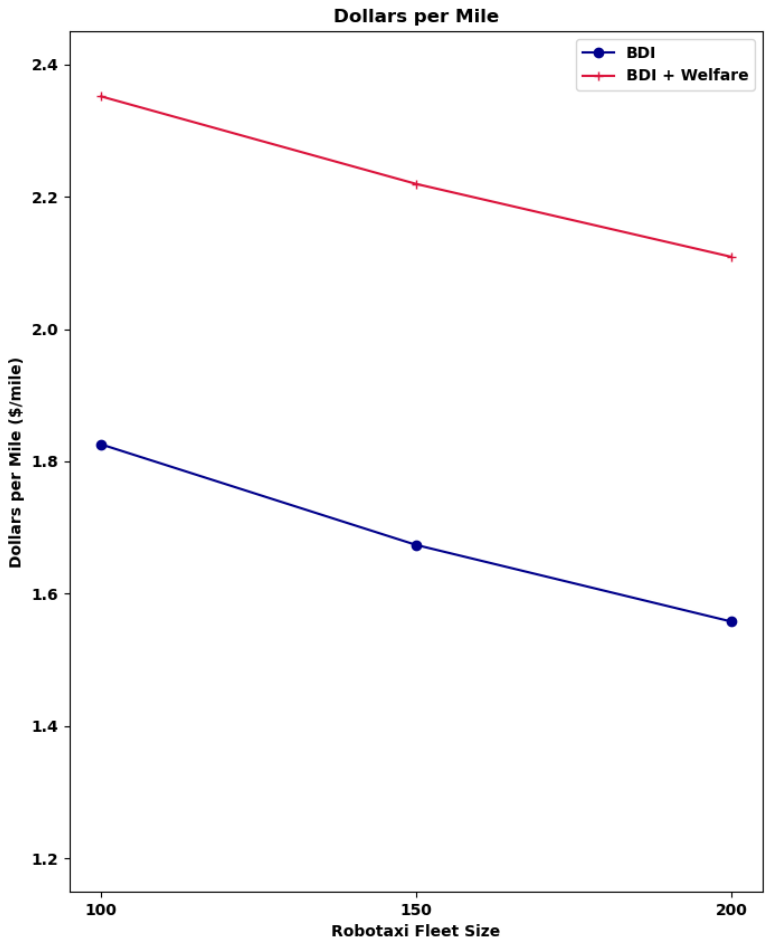

Figure 4.11: Dollars per Mile results for varying robotaxi fleet size: BDI model with and without Welfare Maximization shown.

Once again, these results show a significant improvement for the BDI model when Welfare Maximization is enabled. The Service Rate goes up by about 6%, the Battery-use Efficiency up by 15%, and the Dollars per Mile up by about $0.55 per mile across all fleet sizes.

The BDI model reaches over a 59% Service Rate when Welfare Maximization is enabled. This means the BDI model + Welfare Maximization combined approach could service almost 6 out of every 10 taxi passengers in Chicago with just a 200-vehicle fleet.

Chen et al. found that an "efficiency analysis of the Chicago taxi fleet shows that, for most taxis, around 50% of their in-service time and travel distance are unproductive" [22]. Using this as a signpost, the Battery-use Efficiency results of around 55% for the combined approach indicate our simulated robotaxi fleet is also more efficient on average than a regular Chicago taxi driver.

4.3.4 Integrated Results

Figures 4.12 and 4.13 show the integrated results for all six dispatch approaches under test. Results are shown for the general experiments with fleet sizes of 100, 150 and 200 vehicles.

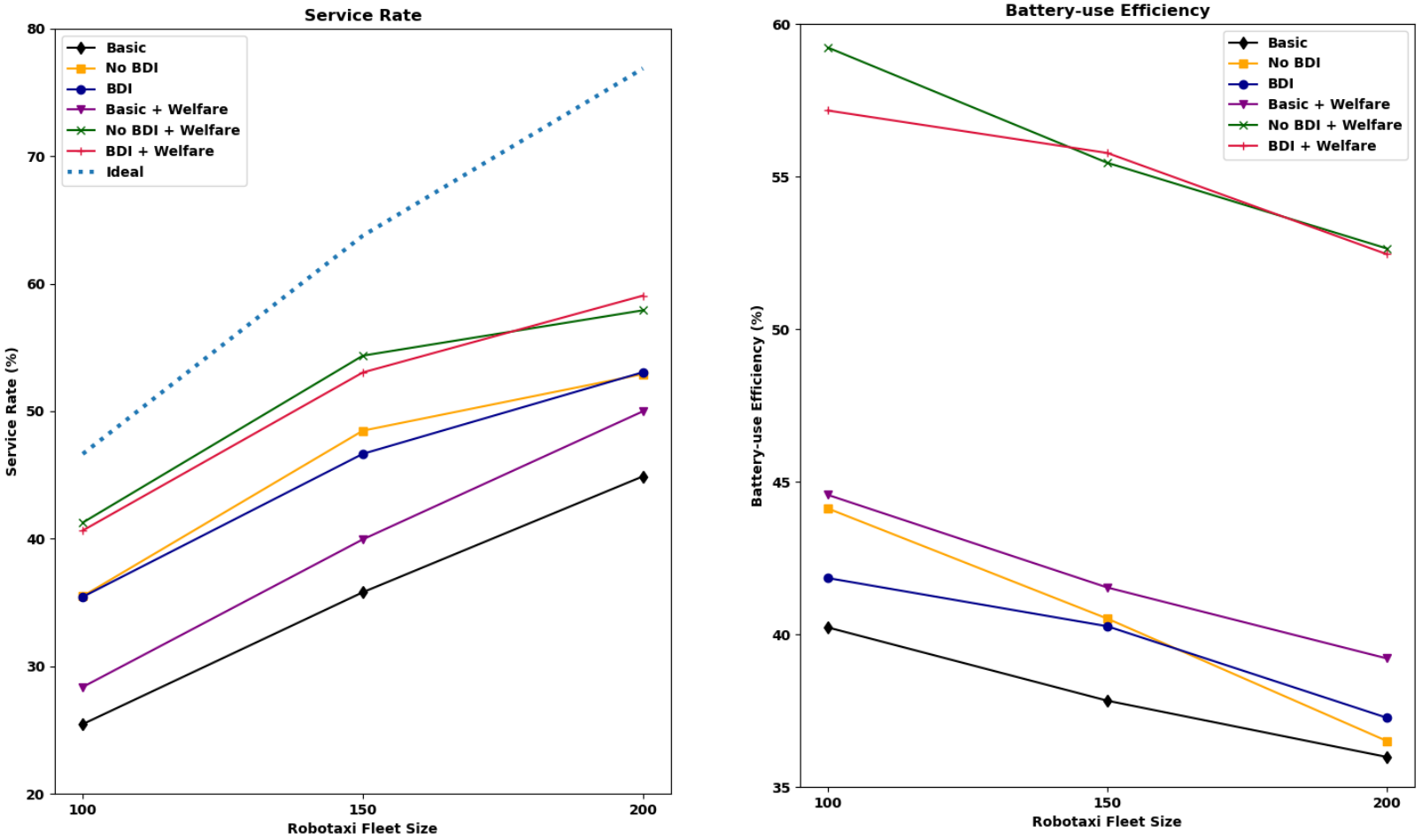

Figure 4.12: Service Rate and Battery-use Efficiency results for varying robotaxi fleet size: Basic approach, No BDI model, and BDI model with and without Welfare Maximization shown.

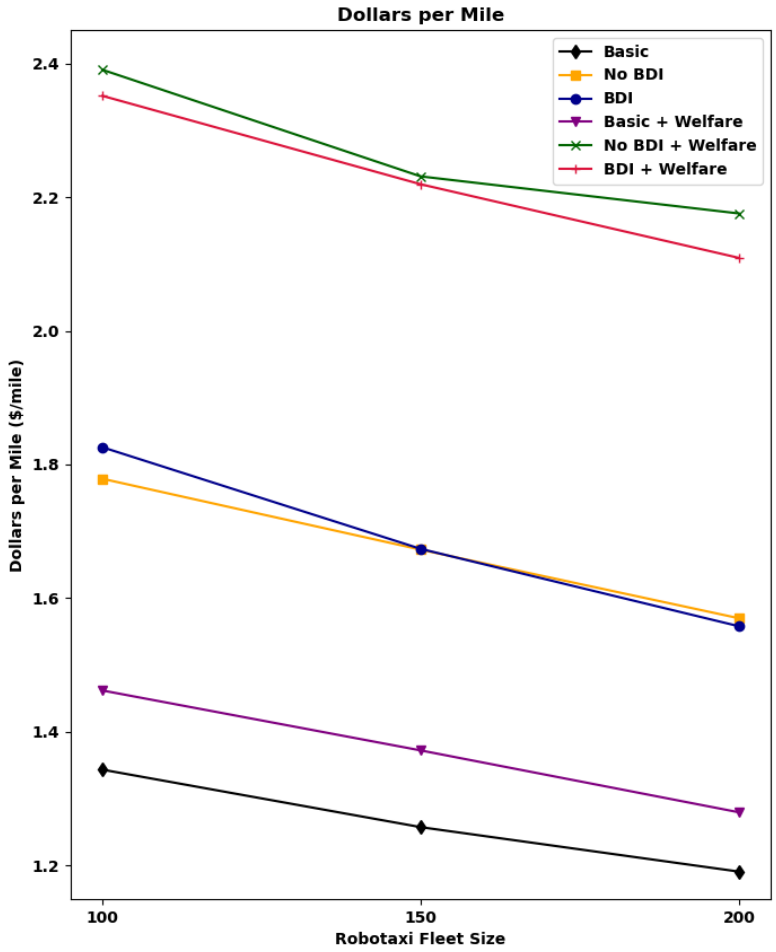

Figure 4.13: Dollars per Mile results for varying robotaxi fleet size: Basic approach, No BDI model, and BDI model with and without Welfare Maximization shown.

These integrated results show the same trends seen in the previous analyses. Both Reinforcement Learning methods, the BDI and the non-BDI model, performed well and had higher metrics results than the Basic approach baseline. This indicates that our modified Deep-Q Learning was able to learn helpful action valuation strategies for the robotaxi agents. This also shows that even with a very simple reward structure, robotaxis within a fleet can us RL to learn to coordinate and improve the reach and efficiency of an AMoD service.

By combining dispatch methods with the Welfare Maximization matching procedure, the most significant improvements in performance metrics are achieved. Welfare Maximization elevates the Basic, No BDI, and BDI dispatch approaches. Significant enhancements can be seen when the two RL models enable Welfare Maximization, versus when it is disabled. There is an especially significant jump in performance for the Battery-Use Efficiency and Dollars per Mile metrics. This shows that Welfare Maximization would be valuable to implement in a real-life AMoD service, where revenue per mile is a defining characteristic in how successful a commercial service is considered.

The BDI model is more robust to demand reduction, but both the BDI and No BDI models show similar results during the general experiments. The performance of both models shows that even with a very simple reward structure, robotaxis can use RL to learn coordination behaviors that improve the reach and efficiency of an AMoD service. Welfare Maximization showed its strength in the results as well, with overall coordination, efficiency, and cost improvements across the board.

4.4 Future Work

4.4.1 Variable Action Set Deep-Q Learning

We proved it is possible to successfully train a robotaxi dispatch model using our Variable Action Set (VAS) version of Deep-Q Learning, which uses action parameters as part of the Neural Network (NN) input. This type of Reinforcement Learning could be extended to include new action types and could be used environments other than just robotaxi dispatch for other tasks with a discrete, variable-sized action set with different action attributes.

Within the realm of robotaxi dispatch, the NN could be trained in a simulation with different scenario parameters. For example, if a company had a specific robotaxi battery size or a different Ops Depot location, the RL training could be used to find the optimal actions for that specific use case.

Future work could also add more constraints and rewards in relation to the actual fare of a trip. This type of training could focus primarily on increasing the profitability of a robotaxi service and could be used to verify the service makes more money than it spends by using the Dollars per Mile metric.

Limiting available Charging Stations

One weakness of this work is the inability to differentiate between robotaxis that are actively charging at the Ops Depot versus those that are just parked there during periods of low rider demand. A new feature could be added to RiderSim to keep track of which robotaxis are actively charging and how many charging stations are left open, if there is a limited number. The number of available charging stations could then be included as an input to the VAS Deep-Q learning NN, which would allow the robotaxi fleet to learn strategies for coordinating in the Ops Depot when there are a limited number of charging stations that must be shared.

4.4.2 Real-life Implementation Details for Welfare Maximization

Our results show that Welfare Maximization is likely a beneficial addition to any dispatch method. More investigations into the details of actually implementing a Welfare Maximization system in real life would be helpful. For example, robotaxis do not all complete their missions simultaneously every 15 minutes. At what interval should Welfare Maximization matching happen in a real-life system? How many riders and robotaxis should be considered within the same matching area? If a robotaxi has not yet completed its mission, but will be arriving soon, should that robotaxi be included in the matching process? What action selection repair mechanisms could be utilized if a robotaxi matched to a trip fails to arrive or has an issue before reaching the rider? Does Welfare Maximization matching need to be performed by a central dispatcher, or could it be decentralized to the level of individual robotaxi agents coordinating through more localized communication?

These and many other questions would need to be answered to implement a Welfare Maximization system for a real-life robotaxi fleet.

4.4.3 Robotaxi Dispatch Simulation Use-cases

As AMoD services continue to grow around the world, robotaxi fleet dispatch simulations like RiderSim will become increasingly useful. Future research possibilities using this type of simulation include the following:

- Simulate road closures – remove connections between selected areas to verify how robust a robotaxi dispatch model is to limited road networks
- Simulate using data from different cities – verify how robust a robotaxi dispatch model is to new road networks and demand patterns
- Simulate large events – verify how robust a robotaxi dispatch model is to an extreme increase in trip requests in a small area
- Simulate expanding the fleet's Geofence – verify how well a robotaxi fleet would perform if the service area were expanded
- Simulate different locations for an Ops Depot – investigate which portion of a city would be best for placing charging stations
- Simulate more or less charging stations – estimate the ideal number of charging stations needed for a specific robotaxi fleet size
- Simulate time-variant fleet sizes – investigate if varying the number of robotaxis throughout the day can improve service efficiency (for example, housing some robotaxis in the Ops Depot during times with lower rider demand)
- Simulate multiple Ops Depots – estimate how much a new depot would improve a robotaxi service and decide if the new depot is worth the cost of bring-up

References

[1] K. Korosec, "Waymo begins testing robotaxis on LA freeways," *TechCrunch*, Jan. 2025. [Online]. Available: https://techcrunch.com/2025/01/28/waymo-begins-testing-robotaxis-on-la-freeways/

[2] D. Lillo et al., "Do Autonomous Vehicles Outperform Latest-Generation Human-Driven Vehicles? A Comparison to Waymo's Auto Liability Insurance Claims at 25.3M Miles," *Waymo Research*, 2024. [Online]. Available: https://waymo.com/safety/research/

[3] Zoox, "Going beyond seeing to perceiving the world around us: An introduction to the Zoox perception system," *Zoox Journal*, Nov. 2023. [Online]. Available: https://zoox.com/journal/perception/

[4] Li, Jiyao, "Optimizing Mobility on Demand Systems: Multiagent Reinforcement Learning Approaches to Order Assignment and Vehicle Guidance" (2024). All Graduate Theses and Dissertations, Fall 2023 to Present. 337. https://digitalcommons.usu.edu/etd2023/337

[5] Zoox, "Las Vegas, let's ride," *Zoox Journal*, Nov. 2024. [Online]. Available: https://zoox.com/journal/las-vegas/

[6] S. Correa et al., "Who's in Charge Here? Scheduling EV Charging on Dynamic Grids via Online Auctions with Soft Deadlines," In Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '20). Association for Computing Machinery, New York, NY, USA, pp. 41–49, 2020. https://doi.org/10.1145/3408308.342761

[7] G. Wang et al., "For ETaxi: Data-Driven Fleet-Oriented Charging Resource Allocation in Large-Scale Electric Taxi Networks" ACM Trans. Sen. Netw. 19, 3, Article 63, Aug. 2023. https://doi.org/10.1145/3570958s

[8] H. Yao, et al., "Deep multi-view spatial-temporal network for taxi demand prediction." *AAAI Press*, In Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, 2018, Article 316, 2588–2595.

[9] Xiaoting Zhou, Lubin Wu, Yu Zhang, Zhen-Song Chen, Shancheng Jiang, "A robust deep reinforcement learning approach to driverless taxi dispatching under uncertain demand," *Information Sciences*, Volume 646, 2023, 119401, ISSN 0020-0255, https://doi.org/10.1016/j.ins.2023.119401

[10] L. Qi, M. Li, X. Guo and W. Luan, "Multi-Objective Optimization for robotaxi Dispatch With Safety-Carpooling Mode in Pandemic Era," in IEEE Transactions

on Intelligent Transportation Systems, vol. 26, no. 1, pp. 878-891, Jan. 2025, doi: 10.1109/TITS.2024.3486152.

[11] Ning Wang, Jiahui Guo, "Multi-task dispatch of shared autonomous electric vehicles for Mobility-on-Demand services – combination of deep reinforcement learning and combinatorial optimization method," Heliyon, Volume 8, Issue 11, 2022, e11319, ISSN 2405-8440, https://doi.org/10.1016/j.heliyon.2022.e11319

[12] G. Fan et al., "Joint Order Dispatch and Charging for Electric Self-Driving Taxi Systems," IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, London, United Kingdom, 2022, pp. 1619-1628, doi: 10.1109/INFOCOM48880.2022.9796825.

[13] City of Chicago, Nov. 2024, "Boundaries - Community Areas (current)," Chicago Data Portal. [Online]. Available: https://data.cityofchicago.org/Facilities-Geographic-Boundaries/Boundaries-Community-Areas-current-/cauq-8yn6

[14] City of Chicago, Jan. 2025, "Taxi Trips (2024-)," Chicago Data Portal. [Online]. Available: https://data.cityofchicago.org/Transportation/Taxi-Trips-2024-/ajtu-isnz/about_data

[15] R. Jonker, T. Volgenant. "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, pp. 325-340, 1987.

[16] City of Chicago, Nov. 2024, "CommAreas," Chicago Data Portal. [Online]. Available: https://data.cityofchicago.org/Facilities-Geographic-Boundaries/CommAreas/igwz-8jzy/about_data

[17] Mebarki et al., "Impact of the Air-Conditioning System on the Power Consumption of an Electric Vehicle Powered by Lithium-Ion Battery", Modelling and Simulation in Engineering, vol. 2013, no. 1, 935784, 2013. https://doi.org/10.1155/2013/935784

[18] Electric Vehicle Knowledge Exchange, "Hyundai Ioniq 5 Long Range AWD charging curve & performance," *EVKX*, Jul. 2025. [Online]. Available: https://evkx.net/models/hyundai/ioniq_5/ioniq_5_long_range_awd/chargingcurve/

[19] James, "Average Electric Car kWh Per Mile [Results From 231 EVs]," *Eco Cost Savings*, Feb. 2024. [Online]. Available: https://ecocostsavings.com/average-electric-car-kwh-per-mile/

[20] H. Newe, S. Deters and R. Saiju, "Analysis of Electrical Charging Characteristics of Different Electric Vehicles Based on the Measurement of Vehicle-specific Load Profiles," NEIS 2019, Conference on Sustainable Energy Supply and Energy Storage Systems, Hamburg, Germany, 2019, pp. 1-6.

[21] J. Knowles, "Could Chicago cab services make a comeback?," *ABC 7 Chicago*, Oct. 2021. [Online]. Available: https://abc7chicago.com/post/chicago-taxi-cab-driver/11184342/

[22] Y. Chen et al., "Characterization of Taxi Fleet Operational Networks and Vehicle Efficiency: Chicago Case Study," Transportation Research Record, 2672(48), pp. 127-138, 2018. https://doi.org/10.1177/0361198118799165